

Efficient Scheduling in Distributed Operating Systems:

A Unified Framework for Local Allocation, Graph-Job Mapping, and Coordinated Routing

Zhao Jia

April 2026

Acknowledgments

This report was prepared for the course CSC6032 at the Chinese University of Hong Kong, Shenzhen. Thanks for the course instructor for guidance and feedback, and acknowledge the original research papers by Belgacem *et al.*, Hosseinalipour *et al.*, and Zhao *et al.* on which this unified paper is based.

Abstract

Modern cloud infrastructures have evolved from single data-center deployments to globally distributed ecosystems. In this geo-distributed setting, the operating system faces three intertwined challenges:

- dynamically allocating heterogeneous virtual-machine resources to arriving independent tasks;
- mapping complex graph-structured jobs onto physical data-center topologies while respecting communication constraints and power budgets;
- routing wide-area network (WAN) traffic so that data transfer does not become the bottleneck.

In this thesis, we propose a unified three-layer framework for efficient scheduling in distributed operating systems. The framework consists of

- a local dynamic resource allocation mechanism based on the Multi-Objective Symbiotic Organism Search (MOSOS) algorithm;
- a power-aware graph-job allocation strategy for geo-distributed cloud networks using cloud crawlers and regret-matching bandits;
- a joint task scheduling and routing system (HPS+) that coordinates computation and wide-area network resources.

For each layer we present the system model, the mathematical formulation, the core algorithmic ideas, and the experimental evidence. Our integrated framework

demonstrates how a distributed OS can move from local optimality toward global, network-coupled scheduling.

Keywords: Distributed operating systems, geo-distributed cloud, dynamic resource allocation, task scheduling, graph jobs, WAN routing, multi-objective optimization.

Contents

Acknowledgments	1
Abstract	2
Contents	5
List of Tables	6
List of Figures	8
List of Algorithms	9
1 Introduction	10
1.1 Motivation: Data-Intensive Geo-Distributed Applications	10
1.2 From Independent Tasks to Graph Jobs	11
1.3 The Network Bottleneck	12
1.4 Research Gaps and Our Scope	12
2 Related Work	14
2.1 Categories of Task Scheduling	16
2.2 Performance-Oriented Scheduling in Geo-Distributed Clouds	20
2.2.1 Computing Resource Utilization	20
2.2.2 Data Management	22
2.3 Fairness and Fault-Tolerance in Scheduling	22
2.3.1 Fairness	23

2.3.2	Fault-Tolerance	23
2.4	Joint Scheduling and Routing	24
2.5	Challenges and Open Issues	24
3	System Model and Problem Formulation	26
3.1	System Architecture	26
3.2	Local VM Pricing and Makespan Model	28
3.3	Data-Center Power and Cost Model	28
3.4	Network Transfer and MIP Formulation	29
4	Proposed Framework	31
4.1	Layer I: Local Dynamic Resource Allocation via MOSOS	31
4.1.1	Dynamic Resource Allocation Model	31
4.1.2	MOSOS Algorithm	34
4.2	Layer II: From Local to Global: Graph-Job Allocation in GDCNs	39
4.2.1	System Model for Graph Jobs	39
4.2.2	Small-Scale GDCNs: Centralized Convex Approach	40
4.2.3	Medium-Scale GDCNs: Distributed CDGA	42
4.2.4	Large-Scale GDCNs: Cloud Crawlers and RMBA	44
4.3	Layer III: Joint Task Scheduling and Routing via HPS+	55
4.3.1	Augmented Hypergraph Model	55
4.3.2	Routing and Bandwidth Allocation (RBA)	58
5	Performance Evaluation	60
5.1	Experimental Results: MOSOS	61
5.2	Experimental Results: Graph-Job Allocation	63
5.3	Experimental Results: HPS+	67
6	Conclusion and Future Work	72
6.1	Implementation Considerations	73
6.2	Conclusion	74
	References	74

List of Tables

2.1	Comparison of different geo-distributed computing environments. . .	16
5.1	Summary of Experimental Settings Across the Three Proposed Methods	60
5.2	Dynamic model evaluation for FCFS, PSO, MOGA, and MOSOS using benchmark datasets.	64
5.3	Scheduling Results on the China-VO Motivational Example	68

List of Figures

2.1	Geo-distributed computing environment with multiple cloud regions, distributed data sources, and a central task scheduler.	15
2.2	Taxonomy of task scheduling in geo-distributed computing.	17
2.3	Green data-center scheduling leverages renewable energy sources (wind, solar) and carbon-aware workload management to reduce environmental impact.	20
3.1	System architecture for distributed job execution in the geo-distributed cloud.	27
4.1	Proposed architecture for dynamic resource allocation in a single cloud cluster.	32
4.2	Flowchart of the MOSOS algorithm.	35
4.3	System model for graph-job allocation in geo-distributed cloud networks with various scales.	41
4.4	Orchestrating the data transfers to balance their completion time. . . .	55
5.1	Performance comparison of MOSOS, MOGA, FCFS, and PBACO on the synthetic task set.	62
5.2	Stability analysis of makespan for MOSOS and MOGA over ten independent runs.	62
5.3	Stability analysis of reservation cost for MOSOS and MOGA over ten independent runs.	63

5.4	Simulation results of the small-scale GDCN: (a) incurred power, (b) cumulative average power, and (c) profit comparison versus greedy baselines.	65
5.5	Evolution of CDGA parameters at six sampled DCs: (a) local variable Λ , (b) global variable γ , and (c) offered slots.	65
5.6	Simulation results of the medium-scale GDCN: (a) incurred power, (b) cumulative average power, and (c) profit comparison versus greedy baselines.	66
5.7	Simulation results of the large-scale GDCN with fixed prices: (a) $P^{90\%}$ convergence, (b) utility comparison, and (c) effect of similarity coefficient and number of clusters.	66
5.8	Simulation results of the large-scale GDCN with dynamic prices: (a) average utility, (b) total power consumption, and (c) reduction in currency circulation.	67
5.9	Total data transfer volume generated by algorithms in the B4 network under different CCR settings (normalized to ≤ 1).	69
5.10	Total data transfer volume generated by algorithms in the CVO network under different CCR settings (normalized to ≤ 1).	69
5.11	Makespan generated by algorithms in the B4 network under different CCR settings (normalized to ≤ 1).	69
5.12	Makespan generated by algorithms in the CVO network under different CCR settings (normalized to ≤ 1).	69
5.13	Comparison between HPS+ and the optimal MIP solution on the CVO network (FileNum = 2000). Top row: file transfer volume; bottom row: makespan.	70
5.14	Running time of HPS+, GS, CDS, HPS, and Flutter on the CVO network.	71

List of Algorithms

1	Dynamic Resource Allocation	33
2	Pareto-Dominance Test	37
3	Multi-Objective Symbiotic Organism Search (MOSOS)	38
3	Multi-Objective Symbiotic Organism Search (MOSOS) (continued) . .	39
4	CDGA: Consensus-Based Distributed Graph-Job Allocation	44
5	Cloud Crawling Routine (CCR)	45
5	Cloud Crawling Routine (CCR) (continued)	46
6	Find_Tot_Cost	47
7	Create_Alloc	47
8	BST_Add	48
9	BRMA: Boosted Regret Minimization Assignment	52
10	RMBA: Regret Matching-Based Assignment	54
11	HPS+ Main Algorithm	57
12	Routing and Bandwidth Allocation Algorithm (RBA)	59

Chapter 1

Introduction

Information technology and the Internet now permeate daily life, from personal communication to industrial management. Cloud computing amplifies this transformation by delivering computing power and storage through remote servers that a large number of customers share dynamically. Enterprises avoid upfront infrastructure costs, focus on core business, and benefit from faster deployment, easier management, and lower maintenance [4, 13, 43].

Virtualization is an integral part of the cloud. It makes resource allocation more reliable, efficient, and dynamic, with high availability and ease of monitoring. Virtualization also allows multiple VMs to share one physical machine, improving performance and keeping services running through hardware failures. However, VMs are configured with different settings, which makes the cloud environment inherently heterogeneous [4]. Resource allocation must satisfy strategic goals for both cloud providers and customers, so task scheduling remains a challenging topic in this domain. Like ordinary scheduling, cloud resource-allocation research seeks an optimal mapping from tasks to resources. Task scheduling in the cloud is NP-hard [4].

1.1 Motivation: Data-Intensive Geo-Distributed Applications

A concrete motivation for geo-distributed scheduling comes from scientific computing. The China Virtual Observatory (China-VO) project, for example, deploys telescopes that receive and generate terabytes of data across remote locations. Process-

ing this data requires extracting information from many wavebands (X-ray, Gamma-ray, infrared, etc.). A native approach that aggregates all data to a single data center (DC) causes massive network congestion and high latency [43]. Therefore, data centers are built close to the telescopes, forming a wide-area distributed network. Geo-distributed data analytics can provide fast and efficient processing by placing compute nodes close to the actual datasets, thus reducing network cost and latency [43].

The China-VO platform serves as a running example throughout this thesis. It includes instruments such as the Shanghai 65-meter radio telescope, the LAMOST optical telescope in Xinglong, and 2.4-meter telescopes in Lijiang. These telescopes produce data for various parameters including electromagnetic radiation, gamma-ray, X-ray, and infrared ray, and must extract useful information from terabytes of data [43]. The dominant approach for data analytics is to aggregate all the data to a single datacenter and process them locally. This degrades analytics timeliness because of the large data volume and low WAN transfer speed. A single datacenter also lacks the compute and storage headroom to keep up with growing, especially data-intensive workloads [43].

1.2 From Independent Tasks to Graph Jobs

In early cloud-scheduling literature, tasks were often assumed to be independent or embarrassingly parallel. Modern big-data frameworks, however, exhibit complicated structures represented by graphs, where nodes denote sub-tasks (each occupying one server slot) and edges indicate communication constraints among them [13]. One common example is receiving data from Twitter and counting the number of times a hashtag is mentioned; each step runs in a processing element, and these elements enforce the separation of each logical step (e.g., receiving updates, extracting hashtags, counting hashtags, ordering hashtag pools) [13].

This graph-job model introduces two new complications. First, finding a feasible allocation between a graph job and a geo-distributed cloud network (GDCN) is similar to the sub-graph isomorphism problem in graph theory, which is NP-complete [13]. Second, execution cost in a GDCN depends mainly on the real-

time power consumption of utilized DCs. An applicable allocation framework must therefore allocate incoming graph jobs quickly while tracking the effect on each DC's power-consumption state [13].

1.3 The Network Bottleneck

Even after a graph job has been mapped onto DCs, the wide-area network (WAN) between DCs remains a bottleneck. Non-local input data generates traffic flows on the WAN. Their endpoints depend on the selected data replica and task placement. Many tasks of the same application often have shared input data, and a dataset could be accessed by multiple tasks simultaneously [43]. The scheduler therefore faces a three-way tension: WAN bandwidth is scarce, datacenter storage is limited, and compute capacity is finite. The distance between tasks and their input data, or the distance among input data sets, makes data routing more complex [43].

1.4 Research Gaps and Our Scope

Most existing works lack emphasis on cloud dynamicity (resource availability and task-arrival variability). Few approaches tackle multi-objective optimization (simultaneous minimization of makespan and power/cost). Conflicts between execution time and resource cost remain unresolved in many single-objective methods. Perhaps most importantly, there is a lack of joint optimization for *computation + power + network* [4, 13, 43].

Our proposed framework spans three layers:

1. **Local dynamic resource allocation** [4]: a dynamic allocation model and the Multi-Objective Symbiotic Organism Search (MOSOS) algorithm that minimize makespan and monetary cost in a single-cluster cloud.
2. **Power-aware graph-job allocation in GDCNs** [13]: a suite of scalable algorithms which centralized convex programming for small networks, consensus-based distributed graph-job allocation (CDGA) for medium networks, and cloud

crawlers with regret-matching bandits (RMBA) for large networks, map graph jobs onto geo-distributed DCs while explicitly modeling DC power consumption.

3. **Coordinated task scheduling and routing** [43]: the HPS+ framework that jointly optimizes task placement and WAN routing via an augmented hypergraph model and a task-aware Routing and Bandwidth Allocation (RBA) algorithm.

By unifying these three lines of work we provide a comprehensive picture of how a distributed OS can move from local resource allocation to global, network-coupled scheduling.

Chapter 2

Related Work

Geo-distributed computing distributes tasks across multiple locations to enable global scalability, leverage computational capacity, and provide geographical redundancy for enhanced reliability. The paradigm also minimizes user-perceived latency by processing data closer to its source, and inherently supports regional data locality requirements that many modern applications demand. However, these characteristics also introduce unique challenges in resource management and data transfer that traditional centralized scheduling systems do not encounter.

Researchers have developed numerous scheduling algorithms tailored for geo-distributed computing systems, aiming to reduce overall makespan, minimize data transfer costs, or ensure fairness and fault tolerance. These approaches incorporate a range of techniques including heuristics, mathematical programming, AI-driven methods, and hybrid combinations. In this chapter we review the most relevant strands of literature, organized first by scheduling technique and then by the core objectives of performance, fairness, and fault tolerance [32].

Table 2.1 summarizes the key differences among the four major geo-distributed computing environments. Geo-distributed cloud computing (GDCC) offers virtually unlimited workload capacity and high scalability, but at the cost of high response latency and bandwidth demand. Edge computing (EC), by contrast, targets real-time latency-sensitive tasks with limited resources. Cloud-edge computing (CEC) bridges the two, while geo-distributed supercomputer computing (GDSC) delivers excep-

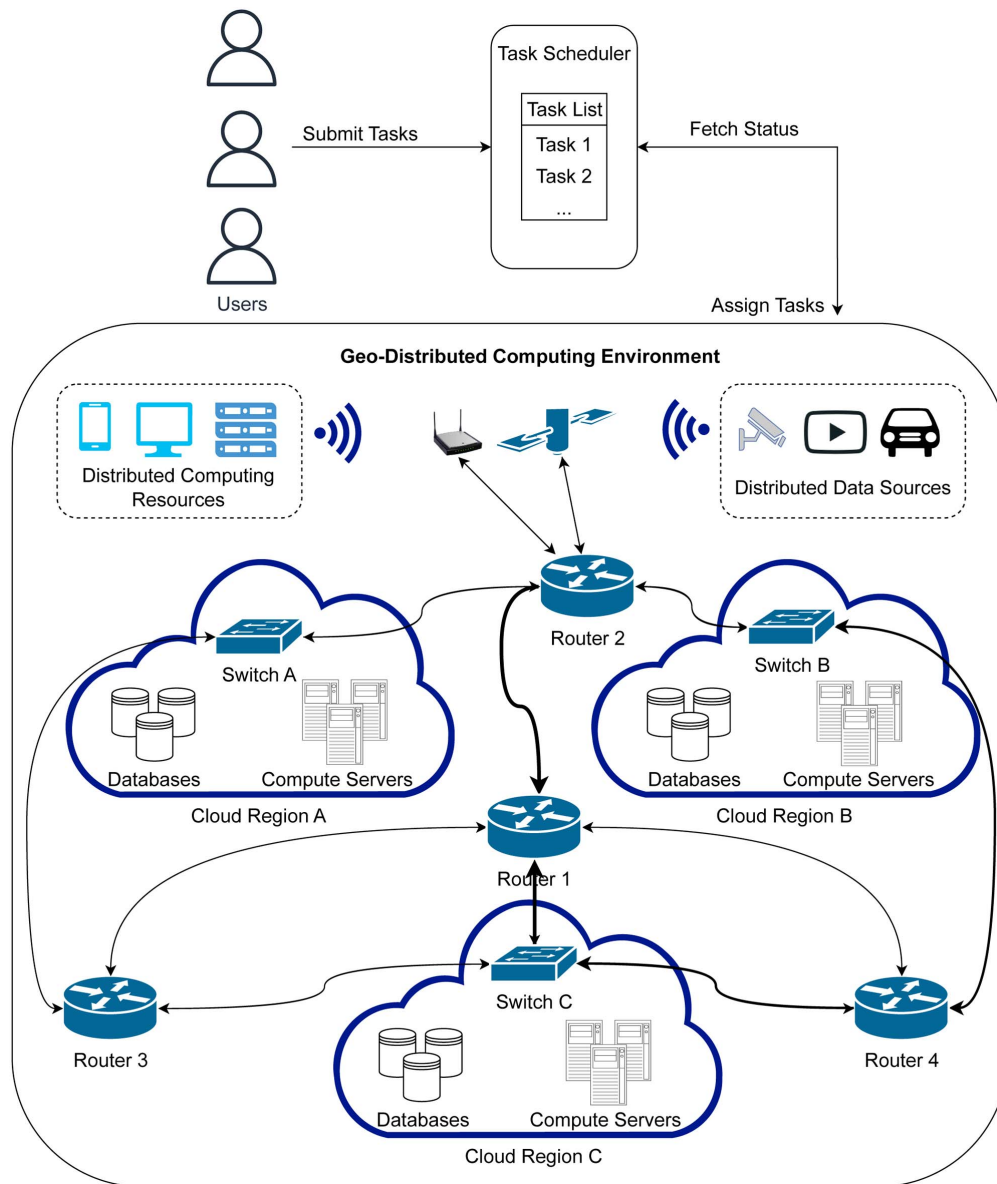


Figure 2.1: Geo-distributed computing environment with multiple cloud regions, distributed data sources, and a central task scheduler.

tional performance for scientific workloads. Because this thesis focuses on GDCC, the algorithms we review and propose must contend with high latency, heterogeneous bandwidth, and general-purpose task types.

Table 2.1: Comparison of different geo-distributed computing environments.

Feature	GDCC	EC	CEC	GDSC
Respond Latency	High Latency	Moderate Latency	Low Latency	Extremely Low
Workload Size	Virtually Unlimited	Relatively Small	Moderate Workload	Exascale, Heavy
Performance	High, Scalable	Low, Limited	Moderate	Exceptional, Scalable
Bandwidth	High Demand	Low Demand	Reduced Demand	Very High Demand
Task Type	General-Purpose (E.g., Web Services)	Real-Time, Latency-Sensitive	Latency-Sensitive & Compute-Intensive	Scientific Computing, Large-Scale

*GDCC: Geo-Distributed Cloud Computing.

*EC: Edge Computing. CEC: Cloud-Edge Computing.

*GDSC: Geo-Distributed Supercomputer Computing.

2.1 Categories of Task Scheduling

Fig. 2.2 gives a taxonomy of task scheduling in geo-distributed computing. The four main branches are geo-distributed cloud computing, edge computing, cloud-edge computing, and geo-distributed supercomputer computing. Within each branch, scheduling methods are further divided by objective (performance, fault tolerance, fairness) and by technique (heuristic, AI-based, mathematical, hybrid). In this thesis we focus primarily on the geo-distributed cloud-computing branch, because it is the setting addressed by all three of our proposed layers.

Cloud task-scheduling schemes fall into four broad technique groups: heuristic scheduling, meta-heuristic scheduling, hybrid scheduling, and learning-based scheduling.

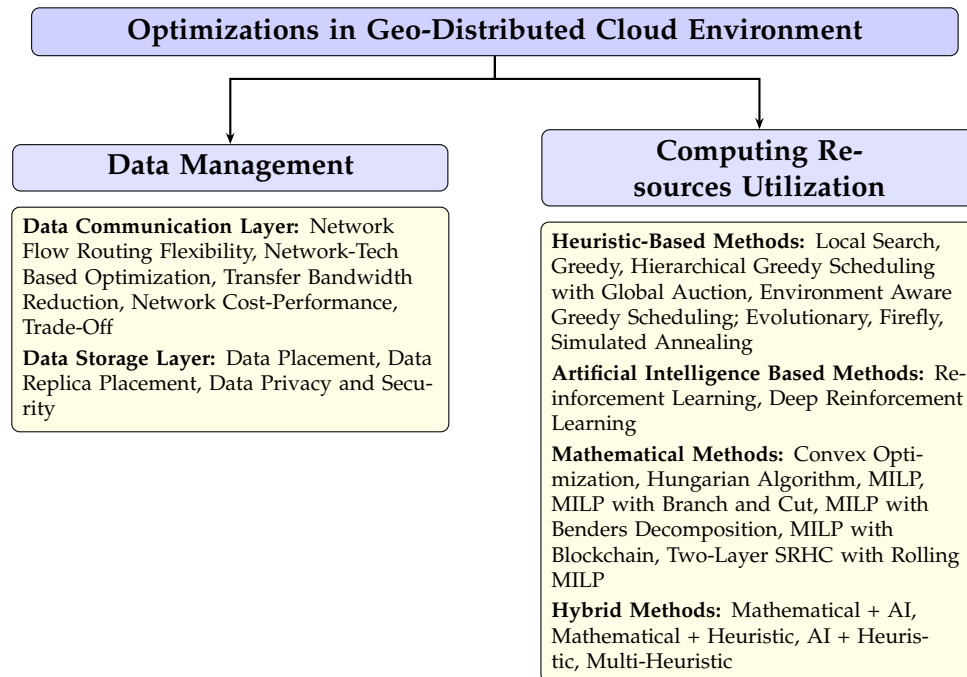


Figure 2.2: Taxonomy of task scheduling in geo-distributed computing.

Heuristic scheduling. These are intuitive, rule-based strategies that rely on greedy principles or simple priority rules to generate feasible solutions quickly. Examples include First-Come-First-Served (FCFS) and the Greedy Scheduling algorithm that assigns tasks to the data center with maximum data locality. In geo-distributed settings, more sophisticated heuristics have been developed. DEWS employs Variable Neighborhood Descent (VND) to swap in-layer tasks and select geo-distributed DCs through three neighborhood structures, integrating Dynamic Voltage and Frequency Scaling (DVFS) for energy optimization. ECWSD adopts a two-phase generate-and-optimize paradigm: an initial task sequence is produced via Heterogeneous Earliest Finish Time (HEFT), followed by refinement using an Adaptive Local Search (ALS) strategy. For straggler mitigation, speculative execution strategies greedily place task replicas on the strongest nodes. Hierarchical greedy frameworks such as MAST introduce a three-level scheduler using distributed auction-based competition among regional schedulers to achieve global optimization in hyperscale environments. While fast, pure heuristics often fail to reach globally optimal solutions and can become trapped in local optima [1,7,9,14,19,22,29].

Meta-heuristic scheduling. These are general-purpose optimization frameworks inspired by natural phenomena or biological behaviors. They explore the solution space efficiently to find near-optimal results. Representative examples include Particle Swarm Optimization (PSO), the Multi-Objective Genetic Algorithm (MOGA), and the Symbiotic Organism Search (SOS) algorithm. Beyond these classics, the literature offers many variants tailored to geo-distributed clouds. IMEAD decomposes revenue-energy cost optimization into sub-problems solved via genetic operators. SPEA-II handles request dispatch and resource allocation under smart grid dynamics. Firefly-inspired methods such as MM-MGSMO use glowworm swarm optimization with MapReduce-based allocation for bandwidth, storage, energy, and computation costs. Simulated-annealing variants (SADE, SBDE, ASBD) integrate the Metropolis criterion with adaptive differential evolution to balance response time and energy cost. Meta-heuristics can explore larger search spaces than pure heuristics, but they require more computation and may converge prematurely [3,17,26,35–38].

Learning-based scheduling. These use machine learning or data-driven techniques to learn scheduling patterns from historical data, enabling adaptive decisions. Reinforcement Learning (RL) has proven particularly effective in dynamic and uncertain environments. RLCut uses multi-agent RL for adaptive graph partitioning that considers network bandwidth heterogeneity. MARL combined with Soft Actor Critic (SAC) schedules AIGC workloads, optimizing GPU utilization while minimizing costs and carbon emissions. Deep RL approaches such as IDQN enable agents to select optimal green DCs and servers to maximize rewards and reduce rejection rates. Proximal Policy Optimization (PPO) has been applied to workload shifting and cloud-bursting in hybrid multi-cloud environments to maximize renewable energy utilization. Learning-based methods excel in dynamic environments but need extensive training data and can be opaque [5,41,42,44].

Mathematical scheduling. These formulate the scheduling problem as an optimization program and solve it with exact or approximate algorithms. Convex optimization has been used to decompose workloads into green and brown components using queuing models, with convexity proved for profit maximization. The Hungarian algorithm assigns map tasks to idle containers by minimizing a data-locality cost matrix. Mixed Integer Linear/Nonlinear Programming (MILP/MINLP) formulations are solved via Branch and Cut, Branch and Bound, Benders Decomposition, and Two-Layer Stochastic Receding Horizon Control. Mathematical methods offer interpretable, constraint-guaranteed solutions but often struggle with scalability as the problem size grows [18,23].

Hybrid scheduling. These combine two or more distinct algorithms to leverage complementary strengths. Mathematical-plus-AI hybrids combine Lyapunov optimization with Generalized Benders Decomposition and DQN for joint energy optimization across servers and network traffic. GT-DRL integrates non-cooperative game theory with DRL for carbon-aware AI inference scheduling. Mathematical-plus-heuristic approaches combine convex programming with cloud crawler-based sub-graph extraction for graph-structured tasks, or hypergraph partitioning with the Dijkstra algorithm. AI-plus-heuristic hybrids such as Turbo use LASSO and GBRT to predict query execution time to guide greedy policies. Multi-heuristic combinations

such as PS3 use Genetic-Simulated-Annealing-based Particle Swarm Optimization (GSPSO). Hybrid methods increasingly dominate the literature because they marry the rigor of mathematical programming, the adaptability of learning, and the speed of heuristics [12, 13, 20, 30, 31, 39].

2.2 Performance-Oriented Scheduling in Geo-Distributed Clouds

Performance is the most widely studied scheduling objective. In the geo-distributed cloud context it splits naturally into two sub-problems: computing resource utilization and data management.

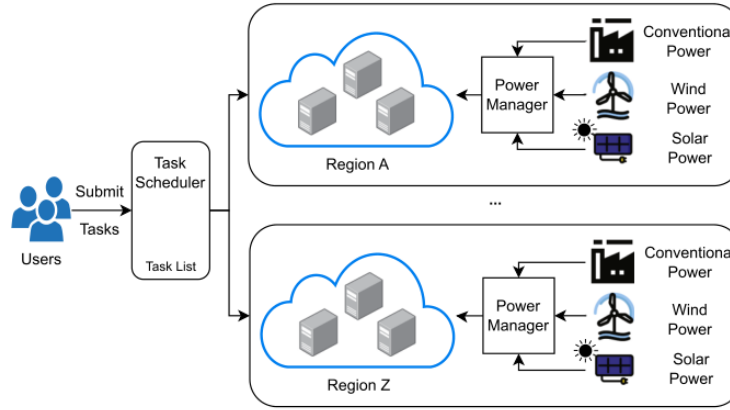


Figure 2.3: Green data-center scheduling leverages renewable energy sources (wind, solar) and carbon-aware workload management to reduce environmental impact.

2.2.1 Computing Resource Utilization

Scheduling methods that target resource utilization can be further classified by the technique they employ.

Heuristic-based utilization. Beyond the general heuristics discussed in Section 2.1, several works focus specifically on computing-resource efficiency. Hierarchical greedy scheduling with global auction (MAST) scales to hyperscale environments by letting regional schedulers compete through distributed auctions. Environment-

aware greedy scheduling (UNL) categorizes renewable-energy uncertainty levels to optimize energy allocation, cost, and carbon emissions. Spatio-thermal-aware workload management greedily selects the lowest-cost DCs based on cooling efficiency and temperature zones. For straggler mitigation, two-phase speculative execution evaluates cluster load to identify straggler-affected jobs and then greedily chooses nodes with the highest computing power, storage capacity, and memory resources to execute replicas. Real-Time Scheduling using Task Duplication (RTSATD) selects tasks with minimum earliest start time while duplicating precursor tasks to the same instance [7].

Nature-inspired utilization. Evolutionary algorithms such as IMEAD, SPEA-II, and EMO decompose multi-objective problems into sub-problems solved via genetic operators, handling smart-grid dynamics and federated-cloud rental costs. Firefly algorithms (MM-MGSMO, mFA) schedule applications in green DCs with adaptive randomization to maximize renewable energy usage. Simulated-annealing variants (SADE, SBDE, ASBD) balance response time and energy cost by integrating the Metropolis criterion with adaptive differential evolution and elite archives [36–38].

AI-based utilization. Reinforcement learning excels in dynamic geo-distributed settings. RLCut uses multi-agent RL for adaptive graph partitioning that respects bandwidth heterogeneity. MARL with SAC schedules AIGC workloads, optimizing GPU utilization while minimizing costs and carbon emissions. Deep RL variants such as IDQN and PPO have been applied to green-DC selection, workload shifting, and cloud-bursting in hybrid multi-cloud environments [5, 42].

Mathematical utilization. Convex optimization formulations decompose workloads into green and brown components and prove convexity for profit maximization. The Hungarian algorithm solves task-to-container assignment by minimizing data-locality cost matrices. MILP and MINLP formulations are tackled via Branch and Cut, Branch and Bound, Benders Decomposition, and rolling-horizon stochastic control [18, 23].

2.2.2 Data Management

Efficient data management is equally important because WAN bandwidth is often the bottleneck in geo-distributed execution.

Data communication. Several works optimize the network layer. Linear-programming formulations split and route coflows across multiple paths. Smart Coflow introduces endpoint flexibility to dynamically adjust flow destinations. HPS+ uses augmented hypergraph partitioning with Routing and Bandwidth Allocation (RBA) to coordinate transfers and computation simultaneously. Congestion-aware allocation (CONA) employs matrix-based traffic allocation and link grading to maximize profit. Software Defined Networks (SDN) enable centralized multi-path routing and worker-node selection in stream analytics. Cross-DC WAN bandwidth reduction is achieved via project migration, table replication, and job outsourcing (Yugong). Cost-aware streaming heuristics (Geo-Cut) partition graphs to minimize inter-DC data movement [45].

Data storage. Data placement optimization refines classical methods (LP, Lagrangian relaxation) with heuristics such as Floyd’s algorithm and ant colony optimization. SpeCH uses hypergraph spectral clustering to model multi-dimensional data associations. Replica placement balances data locality with transfer costs; GEODIS uses LP and heuristics, while DLO-migrate fetches data for non-local tasks using idle bandwidth and DLO-predict periodically replicates predicted hot files. GeoCol uses RL agents to determine replica existence and type, splitting requests via SARIMA-based latency prediction. Security and privacy concerns are addressed by hybrid deep-learning frameworks with Siamese training to prevent secondary data inference, and by privacy-centric process mapping that enforces multi-level data privacy regulations before optimizing communication-aware placement [27, 46].

2.3 Fairness and Fault-Tolerance in Scheduling

While performance dominates the literature, fairness and fault tolerance are equally critical in production geo-distributed systems.

2.3.1 Fairness

Fairness can be defined over different dimensions: job completion time, profit distribution, or resource share.

Max-min fairness formulates the problem as lexicographical minimization and leverages the totally unimodular property to transform it into an equivalent LP problem, significantly improving worst-case job completion time over default Spark schedulers. Profit fairness adopts a two-phase model that prioritizes Cloud Service Providers (CSPs) with below-average profits in an “Essential Selling” status before evaluating candidates via a multi-objective fitness function considering geo-distance, file dependency, and resource price. In edge computing, dynamic Nash bargaining games ensure long-term proportional fairness via Lyapunov-based proportional-fairness resource scheduling decomposed into single-slot sub-problems solved by block-coordinate descent [24].

2.3.2 Fault-Tolerance

Fault-tolerant scheduling involves two main aspects: proactive fault prevention and reactive fault recovery.

Predictive scheduling uses a gray Markov model for resource prediction and factors in error-induced overhead to mitigate anomalies [33]. Approximate Dynamic Programming (ADP) employs a virtualized monitoring model to predict server health and avoid scheduling tasks on unhealthy servers [40]. Load-aware mechanisms switch between proactive task cloning under light load and reactive anomaly detection under heavy load, followed by TSPT algorithm placement optimizing deadline, computing power, and energy consumption [21]. In edge environments, dynamic model partitioning (FTPipeHD) extends GPU-based pipeline parallelism to edge devices, adapting to varying capacities and using mixed weight replication for quick recovery [8]. Checkpointing combined with active replication of high-risk operators alongside RL-based dynamic scaling ensures resilient, low-latency stream processing [34].

2.4 Joint Scheduling and Routing

Prior art in the direction of geo-distributed data analytics often does not consider joint data and compute resource optimization. Hypergraph Partition-based Scheduling (HPS) assigns tasks to data centers utilizing a hypergraph-partition method, which partitions the task graph into balanced groups whose number is equal to the number of data centers [43]. Balanced partitions lead to jagged completion time because data-center capacities differ. Hence HPS reassigns tasks from the long-run data center to the short-run data center to balance the completion time, at the cost of increasing the total file-transfer volume. Flutter formulates the scheduling problem as a lexicographical min-max integer linear programming problem, but assumes that all tasks require the same amount of resources and that network bandwidths are stable over time [43]. These limitations motivate the HPS+ framework that we present in Section 4.3.

2.5 Challenges and Open Issues

Despite the progress surveyed above, several critical challenges remain unresolved.

Security and privacy. Geo-distributed tasks often involve sensitive data across multiple jurisdictions with varying regulations such as GDPR and HIPAA, particularly in federated learning and healthcare. Existing security mechanisms are limited by edge device resource constraints, the complexity of enforcing consistent policies across regions, and cryptographic overhead in real-time applications. Emerging industry solutions such as Google’s Confidential Computing and Trusted Execution Environments (TEEs) protect data during processing on untrusted nodes, yet security-aware scheduling algorithms that natively incorporate regional compliance requirements remain scarce [11].

Emerging workload diversity. Traditional scheduling approaches for HPC and web services are insufficient for emerging workloads such as AI-generated content (AIGC), LLM inference, and AR/VR-integrated intelligent assistants. These workloads require multi-modal task coordination, cross-region computational coor-

dination, and heterogeneous hardware (CPUs, GPUs, accelerators). LLM inference presents unique challenges due to its auto-regressive nature, KV Cache statefulness, and need for low response latency. Most existing geo-distributed scheduling approaches do not adequately address stateful KV caching, resource-demand imbalances across regions, and multi-stage processing pipelines under network dynamics [6].

Next-generation geo-distributed computing. Quantum computing offers potential for solving combinatorial scheduling problems via Quadratic Unconstrained Binary Optimization (QUBO) models and quantum annealing devices. Quantum-inspired heuristic algorithms have already shown improvements in reducing waiting time and maximizing resource utilization on classical computers. Quantum communication via entanglement and quantum key distribution can enable ultra-secure data transmission. However, current mainstream approaches are hybrid models where classical computers coordinate small-scale quantum processors, due to qubit sensitivity to environmental noise and the early stage of fault-tolerant quantum computing [2, 10, 15, 16, 25, 28].

Chapter 3

System Model and Problem Formulation

Before presenting the algorithms, we introduce the system architecture, power models, and cost models that are common to the three layers of scheduling.

3.1 System Architecture

The overall architecture for distributed job execution in a geo-distributed cloud can be conceptualized as a two-tier control plane plus a monitored data plane. The top tier contains a central controller that is responsible for global scheduling decisions. This controller is further decomposed into a task scheduler and an SDN controller. The task scheduler periodically wakes up when new jobs arrive, assembles multiple jobs into a single scheduling batch (sometimes called a “big job”), and computes a global schedule based on the current requirements of the jobs and the state of the underlying data centers. The SDN controller, on the other hand, is responsible for network-wide bandwidth management. It distinguishes between interactive flows, whose demands are predicted using deep-learning models such as LSTM networks, and non-interactive flows generated by big-data analytics applications. For interactive flows the SDN controller proactively reserves bandwidth; for non-interactive flows it relies on a flow-routing module that computes path capacities and makes per-flow routing decisions after the task scheduler has determined replica selection

and task placement.

At the bottom tier, each data center hosts local resource monitors. These monitors track on-going tasks, measure machine resource usage (CPU, memory, disk I/O), and send periodic “heartbeat” messages to the central controller. The heartbeat reports enable the controller to maintain an up-to-date view of cluster health and available capacity. In addition, the controller obtains raw network status (link bandwidths, switch queue occupancies) from OpenFlow-enabled switches deployed across the WAN. By combining application-level telemetry from the monitors with network-level telemetry from the switches, the controller can make informed decisions that co-optimize computation, storage, and communication resources, as illustrated in Fig. 3.1.

ON PARALLEL AND DISTRIBUTED SYSTEMS, VOL. 31, NO. 2, FEBRUARY 2020

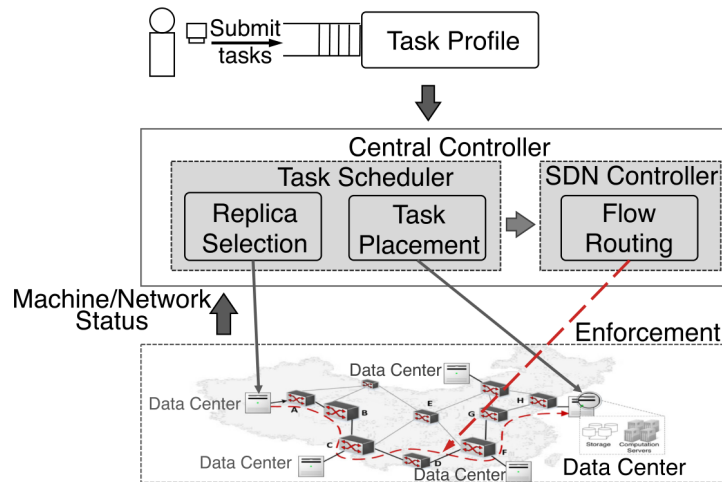


Figure 3.1: System architecture for distributed job execution in the geo-distributed cloud.

3.2 Local VM Pricing and Makespan Model

In the local dynamic-allocation model of [4], a stream of N independent tasks $T = \{T_1, \dots, T_N\}$ arrives at a single cloud cluster. A set of M heterogeneous VMs $VM = \{VM_1, \dots, VM_M\}$ is available. Each VM j is characterized by its CPU speed $MIPS_j$, RAM size $S_{ram,j}$, number of CPUs $S_{cpu,j}$, and hourly price c_j . A binary decision variable $o_{ij} \in \{0, 1\}$ indicates whether task T_i is assigned to VM_j .

The makespan $X(t)$ of an allocation round is the maximum finish time of the last running task among the selected tasks:

$$X(t) = \max_{i,j: o_{ij}=1} \frac{l_i}{MIPS_j}, \quad (3.1)$$

where l_i is the length of task T_i in instructions. The total monetary cost $Y(t)$ is the sum of per-task costs:

$$Y(t) = \sum_{i=1}^N Y_i(t), \quad Y_i(t) = c_j \cdot t_j + \#_{trij}, \quad (3.2)$$

where t_j is the execution duration on VM_j and $\#_{trij} = \frac{F_i}{MIPS_j \times 32} \times 0.023$ captures the transfer-rate cost for a 32-bit bus (F_i is the file size in bits). The multi-objective optimization problem is then

$$Z = \min[X(o), Y(o)], \quad (3.3)$$

subject to VM RAM/CPU capacity constraints and the condition that the number of tasks loaded on a VM does not exceed its CPU count.

3.3 Data-Center Power and Cost Model

In geo-distributed settings, [13] models the power consumption upon utilizing s_i slots of data center d_i comprising N_i cloud servers each with idle power P_i^{idle} as

$$P_i(s_i) = h_i N_i \left(\sigma_i \left(\frac{s_i}{|S_i|} \right)^{a_i} + P_i^{\text{idle}} \right), \quad a_i \geq 2, \quad (3.4)$$

where h_i is the Power Usage Effectiveness (PUE), σ_i is a DC-specific constant chosen so that $\sigma_i + P_i^{\text{idle}}$ equals the peak power P_i^{max} of a server in d_i , and S_i is the set of slots in d_i . The incurred cost of executing a graph job of type j allocated according to a feasible mapping vector $\mathbf{M}^j = [m_1^j, \dots, m_{n_d}^j]$ is

$$\text{Cost}(\mathbf{M}^j) = \sum_{i=1}^{n_d} \lambda_i h_i N_i \left[\sigma_i \left(\frac{L_i + m_i^j}{|S_i|} \right)^{a_i} + P_i^{\text{idle}} \right] + \sum_{i=1}^{n_d} \lambda_i n_i m_i^j, \quad (3.5)$$

where L_i is the original load of DC d_i , n_i is the I/O incurred power per slot, and λ_i is the cost-to-power ratio depending on the DC's location and infrastructure design.

The global optimization problem (P1) is to minimize the total incurred cost for all graph jobs while ensuring DC stability ($\sum_j \sum_k m_i^{j(k)} \leq |S_i| - L_i$) and feasibility of the assignment ($\mathbf{M}^{j(k)} \in \mathcal{M}^j$). Identifying the feasible sets $\mathcal{M}^j$ requires solving the NP-complete sub-graph isomorphism problem, and even with known feasible mappings P1 is an NP-hard nonlinear integer program [13].

3.4 Network Transfer and MIP Formulation

In the coordinated routing model of [43], the substrate cloud system is a graph $G_S = (N_S, E_S)$, where N_S is the set of DCs and E_S is the set of WAN links with capacities R_{uv} . The task graph is $G_V = (N_V, E_V)$ with $N_V = N_T \cup N_F$, where N_T is the set of tasks, N_F the set of input files, and E_V the task-file dependencies.

A task starts processing only after all its input files are transferred to local storage. The data-transfer time of file f of size s_f for task i is s_f / b_{fi} , where b_{fi} is the allocated bandwidth. The makespan c is the completion time of the slowest task. The scheduling problem is formulated as a mixed integer program (MIP) with binary variables x_{ij} (task placement), y_{fki} (replica selection), z_{fki}^t (path usage), and continuous variables $b_{fki}^t \geq 0$ (bandwidth on path t). The objective is $\min c$, subject to the following constraints [43]:

1. **Task completion:** $s_f / b_{fi} + p_i \leq c$ for every task i and its input file f , where p_i is the CPU time; this ensures all tasks finish before the makespan.

2. **Link capacity:** $\sum_{f,i} \sum_{t \ni (u,v)} b_{fki}^t \leq R_{uv}$ for every WAN link $(u,v) \in E_S$; total allocated bandwidth cannot exceed link capacity.
3. **DC capacity:** $\sum_i h_i x_{ij} \leq R_j$ for every DC $j \in N_S$; the aggregate CPU demand of tasks placed in a DC cannot exceed its available cores.
4. **Single placement:** $\sum_{j \in N_S} x_{ij} = 1$ for every task i ; each task is assigned to exactly one DC.
5. **Single replica:** $\sum_{f_k \in \{f\}} y_{fki} = 1$ for every task i and input file f ; exactly one replica of each required file is selected.
6. **Correlation constraints:** $b_{fki}^t > 0$ only if both $y_{fki} = 1$ and $x_{ij} = 1$ for the DC j at the endpoint of path t ; bandwidth is allocated only for active replicas and placed tasks.
7. **Domain constraints:** $x_{ij}, y_{fki}, z_{fki}^t \in \{0, 1\}$ and $b_{fki}^t \geq 0$.

Chapter 4

Proposed Framework

We now describe the three algorithmic layers that move a distributed OS from local optimality toward global, network-coupled scheduling.

4.1 Layer I: Local Dynamic Resource Allocation via MOSOS

Before considering global power or WAN routing, a distributed OS must first solve the local resource-allocation problem: how to assign arriving independent tasks to VMs inside a single cluster in a dynamic fashion. Fig. 4.1 depicts the proposed architecture for this layer.

4.1.1 Dynamic Resource Allocation Model

We propose a dynamic resource-allocation model in which resources are allocated as soon as possible and each arrival request is placed in a queue. To get truly dynamic allocation in continuous time, the system is split into a state-reporting part $\alpha(t)$ (monitoring tasks in the queue and those currently running) and a control part $\beta(t)$ (mapping each arriving task to an appropriate VM). The system evolution is captured by $\dot{C}_I(t) = H(\alpha(t), \beta(t))$.

A novel parameter D , the *dynamic time quantum*, determines the waiting time until the next allocation cycle. When the workload is light D shrinks, allowing the scheduler to seize ready VMs quickly; when the workload is heavy D expands, giving the scheduler time to accumulate enough released CPUs and form a larger,

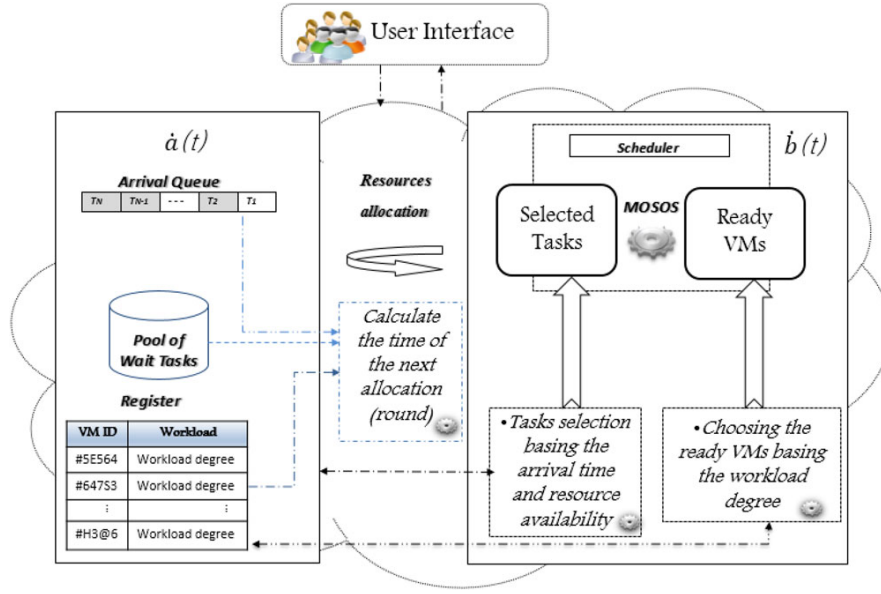


Figure 4.1: Proposed architecture for dynamic resource allocation in a single cloud cluster.

better-matched batch. This trade-off between responsiveness and batch quality is captured by D is computed as

$$D = \frac{e^{-u/d}}{2} X(t), \quad (4.1)$$

where $u = (A + B)/C$ is the arrival-to-running ratio (A = tasks in queue, B = tasks in waiting pool, C = running tasks), d is the availability of resources derived from the workload degree $W_j(t) = (S_{\text{cpu}} - d(t)) \cdot \text{MIPS}_j + S_{\text{ram}} \cdot (1 - \alpha(t))$, and $X(t)$ is the makespan of the previous round.

In each round the scheduler forms a set of selected tasks whose size r equals the total number of CPUs released. A distribution factor g measures task homogeneity. The scheduler selects the first-arrival task first; if the resource availability d is less than g , it moves to the next task and puts the current one in the wait pool.

Algorithm 1 presents the full pseudo-code of the dynamic resource allocation procedure. It repeatedly computes D , forms the ready-VM and selected-task sets,

and invokes MOSOS until all tasks are scheduled.

Algorithm 1 Dynamic Resource Allocation

```

1: Input:  $Alc, A, B, C, t, round$ 
2: Output: Optimized dynamic scheduling
3:  $Alc \leftarrow \text{false}; VM\_List \leftarrow \emptyset; T\_List \leftarrow \emptyset$ 
4: while  $t < \infty$  do
5:    $Alc \leftarrow \text{false}$ 
6:   Calculate  $D$  according to Equation (4.1)
7:   while  $Alc = \text{false}$  do
8:      $t_0 \leftarrow t_0 + 1$ 
9:     if  $t_0 = D$  then
10:       $Alc \leftarrow \text{true}$ 
11:    end if
12:  end while
13:  // Selected tasks
14:  Calculate selected-task size  $r = \sum_{j=1}^M (S_{\text{cpu}} - d_j(t))$ 
15:  for  $i = 1$  to  $r$  do
16:     $T\_List \leftarrow T\_List \cup \{T_i\}$ 
17:    Calculate distribution factor  $g = \frac{1}{r} \sum_{i=1}^r (l_i + F_i)$  and resource availability
     $d$ 
18:    while  $d < g$  do
19:       $T_j \leftarrow \text{next arrival task}$ 
20:      Replace  $T_r$  with  $T_j$  in  $T\_List$ 
21:      Recalculate  $r$  and  $d$ 
22:    end while
23:  end for
24:  // Ready VMs
25:  for each  $VM_j$  do
26:    if  $VM_j$  is ready ( $0 \leq d_j(t) < S_{\text{cpu},j}$  and  $S_{\text{ram},j} \cdot \alpha(t) < S_{\text{ram},j}$ ) then
27:       $VM\_List \leftarrow VM\_List \cup \{VM_j\}$ 
28:    end if
29:  end for
30:  Invoke Algorithm 3 (MOSOS) on  $T\_List$  and  $VM\_List$ 
31:   $t \leftarrow t + 1$ 
32: end while

```

Fitness function. We evaluate each candidate schedule by two conflicting objectives:

makespan and cost. We obtain the makespan $X(t)$, which is the time of executing all selected tasks in each allocation, from

$$X(t) = \max_{i,j: o_{ij}=1} \frac{l_i}{\text{MIPS}_j}, \quad (4.2)$$

and the total cost is

$$Y(t) = \sum_{i=1}^N Y_i(t), \quad (4.3)$$

where the individual task cost is

$$Y_i(t) = c_j \times t_j + \vartheta_{tr_{ij}}, \quad (4.4)$$

and the transfer-rate cost is

$$\vartheta_{tr_{ij}} = \frac{F_i}{\text{MIPS}_j \times 32} \varepsilon, \quad \varepsilon = 0.023. \quad (4.5)$$

Here c_j is the price of using VM_j , t_j is the duration of running task i on VM_j , F_i is the file size occupied by the task, and ε is the transfer price.

4.1.2 MOSOS Algorithm

MOSOS serves as the scheduling engine. It extends the single-objective Symbiotic Organism Search (SOS) meta-heuristic to a bi-objective setting by pairing each symbiotic operator with a Pareto-dominance archive. Fig. 4.2 gives the high-level flow.

Encoding. Each organism is a real-valued vector whose length equals the number of tasks; the entries encode VM assignments. The population is initialized uniformly at random, and the best individual is stored as o_{best} .

Mutualism. Two distinct organisms o_i and o_j exchange information in a way that pulls both toward o_{best} . First, mutual-benefit vectors are computed with random

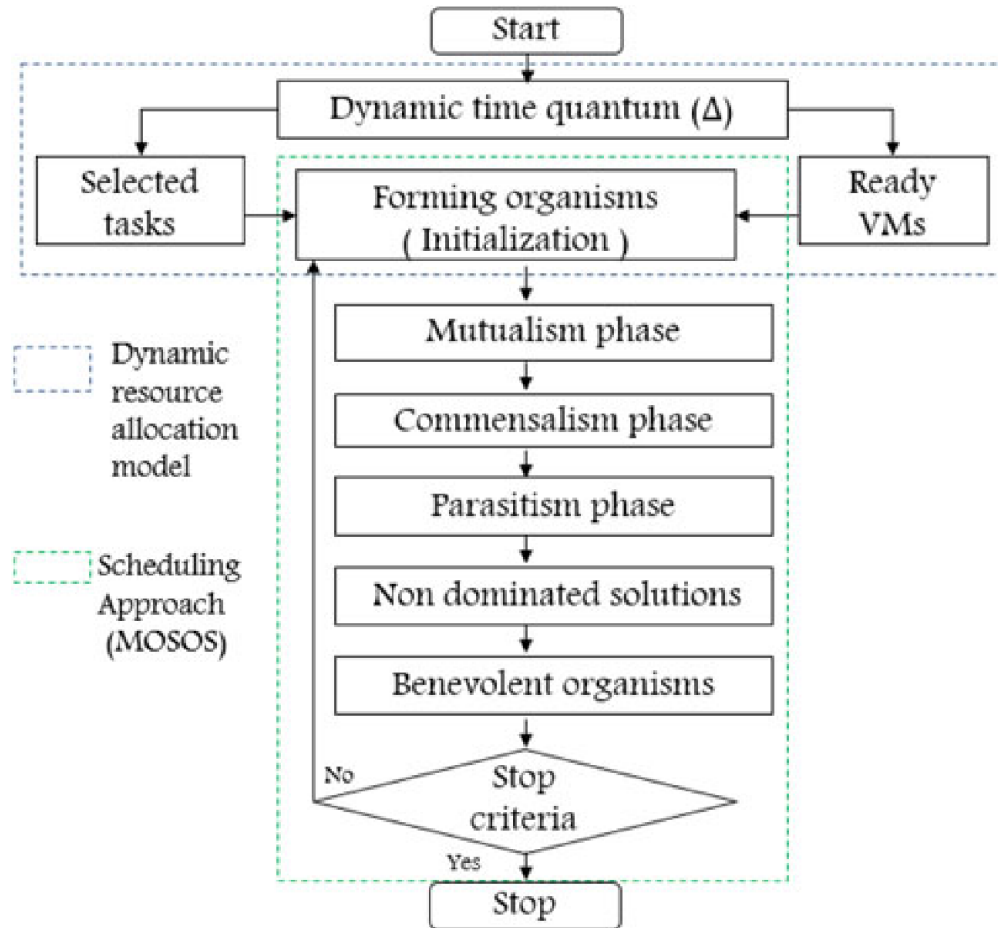


Figure 4.2: Flowchart of the MOSOS algorithm.

mutual factors $g_1, g_2 \in \{1, 2\}$:

$$\sigma_1 = o_i + \mu_1 \left(o_{\text{best}} - \frac{o_i + o_j}{2} g_1 \right), \quad (4.6)$$

$$\sigma_2 = o_j + \mu_2 \left(o_{\text{best}} - \frac{o_i + o_j}{2} g_2 \right), \quad (4.7)$$

where μ_1, μ_2 are uniform random numbers in $[0, 1]$. The updated organisms are then bounded by the number of tasks N :

$$o'_i = \left\lceil \frac{\sigma_1}{N+1} \right\rceil, \quad (4.8)$$

$$o'_j = \left\lceil \frac{\sigma_2}{N+1} \right\rceil. \quad (4.9)$$

A mutant replaces its parent only when it improves fitness.

Commensalism. Organism o_i exploits organism o_j one-sidedly. A relationship vector is computed as

$$\sigma_3 = \mu_3 (o_{\text{best}} - o_i), \quad (4.10)$$

where μ_3 is a uniform random number in $[0, 1]$. The updated organism is wrapped to a valid VM index by

$$o'_i = (\lceil \sigma_3 \rceil \bmod M) + 1. \quad (4.11)$$

Again, replacement is conditional on fitness improvement.

Parasitism. A parasite o_c is generated by scaling a randomly chosen host o_i :

$$\sigma_4 = \mu_4 o_i, \quad (4.12)$$

where μ_4 is a uniform random number in $[0, 1]$. The parasite vector is wrapped to a valid VM index by

$$o_c = (\lceil \sigma_4 \rceil \bmod M) + 1. \quad (4.13)$$

It attacks another organism o_j ; if the parasite is fitter, it displaces the host. This operator injects diversity and helps escape local optima.

Elitism via non-dominated sorting. After the three symbiotic phases, every

organism is compared against an elite archive maintained across iterations using Pareto dominance (Algorithm 2). An organism that dominates an archive member replaces it; otherwise it is discarded. The archive therefore tracks the best non-dominated front discovered so far.

Algorithm 2 Pareto-Dominance Test

Require: Organisms o_i, o_j

Ensure: Dominance flag D

```

1: if ( $X(o_i) < X(o_j)$ ) and ( $Y(o_i) < Y(o_j)$ ) then
2:    $D \leftarrow \text{true}$ 
3: else if ( $X(o_i) < X(o_j)$ ) and ( $Y(o_i) = Y(o_j)$ ) then
4:    $D \leftarrow \text{true}$ 
5: else if ( $X(o_i) = X(o_j)$ ) and ( $Y(o_i) < Y(o_j)$ ) then
6:    $D \leftarrow \text{true}$ 
7: else
8:    $D \leftarrow \text{false}$ 
9: end if return  $D$ 

```

Algorithm 3 spells out the complete MOSOS loop. Each iteration performs the four phases in sequence and then updates the elite archive. The loop terminates after a preset number of iterations or when the archive ceases to change.

The overall scheduling complexity is $O(n^2)$. Task selection and ready-VM detection each cost $O(n)$; MOSOS itself runs in $O(mn)$ for m VMs and n tasks. Because $m \leq n$ in practice, the bound simplifies to $O(n^2)$, lower than MOGA's $O(mn^3)$ for large populations [4].

Algorithm 3 Multi-Objective Symbiotic Organism Search (MOSOS)**Require:** Tasks $T_1, T_2, \dots, T_n$; VMs $VM_1, VM_2, \dots, VM_m$ **Ensure:** An optimized generated schedule

```

1:  $iter \leftarrow 1$ ;  $max\_iter \leftarrow 100$ ;  $O\_list$ ;  $D$  boolean
2: while  $iter \leq max\_iter$  do
3:   for each organism  $o_i$  do
4:      $o_i \leftarrow \text{rand}$ 
5:   end for
6:   // Mutualism phase
7:   for each organism  $o_i$  do
8:     Randomly select  $o_j$  with  $j \neq i$ 
9:     Calculate  $o'_i$  and  $o'_j$  according to Eqs. (4.8) and (4.9)
10:    if  $X(o'_i) > X(o_i)$  then
11:       $o_i \leftarrow o'_i$ 
12:    end if
13:    if  $X(o'_j) > X(o_j)$  then
14:       $o_j \leftarrow o'_j$ 
15:    end if
16:  end for
17:  // Commensalism phase
18:  for each organism  $o_i$  do
19:    Randomly select  $o_j$  with  $j \neq i$ 
20:    Calculate  $o'_i$  according to Eq. (4.11)
21:    if  $X(o'_i) > X(o_i)$  then
22:       $o_i \leftarrow o'_i$ 
23:    end if
24:    if  $X(o_i) < X(o_j)$  then
25:       $o_i \leftarrow o'_i$ 
26:    end if
27:  end for

```

Algorithm 3 Multi-Objective Symbiotic Organism Search (MOSOS) (continued)

```

28:  // Parasitism phase
29:  for each organism  $o_i$  do
30:    Randomly select  $o_j$  with  $j \neq i$ 
31:    Calculate  $o_c$  according to Eq. (4.13)
32:    if  $X(o_j) < X(o_c)$  then
33:       $o_j \leftarrow o_c$ 
34:    end if
35:  end for
36:  // Benevolent phase
37:  for each organism  $o_i$  do
38:    Call Non_dominated_Solutions ( $o_i, o_j, D$ )
39:    if  $D = \text{true}$  then
40:       $o_i \leftarrow o_j$ 
41:    end if
42:  end for
43:   $iter \leftarrow iter + 1$ 
44: end while

```

4.2 Layer II: From Local to Global: Graph-Job Allocation in GD-CNs

Moving from a single cluster's local optimum to a geo-distributed DC network, jobs are no longer independent; sub-tasks have internal communication edges, which means they form a graph. The new bottleneck is inter-DC I/O, power, and latency.

4.2.1 System Model for Graph Jobs

We model the geo-distributed cloud as an aggregated graph $G = (V_S, E_S)$. Each DC d_i contains a set of fully-connected slots S_i ; every slot can communicate with any other slot in the same DC at negligible cost. Inter-DC links form the wide-area edges E_D , so two slots are adjacent either because they sit in the same DC or because their DCs are WAN-connected. The union of all slots $V_S = \bigcup_i S_i$ therefore spans both local and wide-area adjacency [13].

A graph job $G_{\text{job}}^j = (V^j, E^j)$ is a directed acyclic graph whose vertices are sub-

tasks and whose edges are data or control dependencies. Every vertex needs exactly one slot, and every edge demands that its endpoints be placed on adjacent slots. Formally, a feasible allocation is an injective mapping $f_j : V^j \rightarrow V_S$ such that $(v_m, v_n) \in E^j$ implies $(f_j(v_m), f_j(v_n)) \in E_S$. This is precisely the sub-graph isomorphism problem, which is NP-complete [13].

4.2.2 Small-Scale GDCNs: Centralized Convex Approach

For small-scale networks, we tackle P1 through convex relaxation. We relax the discrete constraint on the mapping vectors by allowing $\mathbf{M}^{j,(k)} \in (\mathbb{R}^+)^{n_d}$, leading to the convex optimization problem P3. P3 has a differentiable objective and affine constraints, so it satisfies constraint qualifications and enjoys zero duality gap. We use the Lagrangian dual decomposition method. Let $\lambda = [\lambda_1, \dots, \lambda_{n_d}]$, g , and $\mathbf{L} = [L_1, \dots, L_{n_d}]$ be the Lagrangian multipliers. The Lagrangian function is

$$\begin{aligned} \mathcal{L}(\mathbf{M}, \boldsymbol{\mu}, g, \gamma) = & \sum_{i=1}^{n_d} \lambda_i h_i N_i \left[\sigma_i \left(\frac{L_i + m_i^{j,(k)}}{|S_i|} \right)^{a_i} + p_i^{\text{idle}} \right] + \sum_{i=1}^{n_d} \lambda_i n_i m_i^{j,(k)} \\ & + \sum_{i=1}^{n_d} \mu_i \left(m_i^{j,(k)} - |S_i| + L_i \right) + g \left(\sum_{i=1}^{n_d} m_i^{j,(k)} - |V^j| \right) - \sum_{i=1}^{n_d} \gamma_i m_i^{j,(k)}. \end{aligned} \quad (4.14)$$

The optimal continuous solution admits the closed form

$$m_i^{j,(k)*} = \left[\frac{(\gamma_i^* - \mu_i^* - g^* - \lambda_i n_i) |S_i|^{a_i}}{\lambda_i h_i N_i \sigma_i a_i} \right]^{\frac{1}{a_i-1}} - L_i, \quad (4.15)$$

and the optimal Lagrangian multipliers are obtained by solving the gradient equations via an iterative gradient-ascent algorithm. Once we obtain the continuous solution $\mathbf{M}_g^{j,(k)*}$, we project it back onto the discrete feasible set by solving the weighted mean-square problem

$$\mathbf{M}^{j,(k)*} = \arg \min_{\mathbf{M} \in \mathcal{M}^j} \sum_{i=1}^{n_d} w_i (m_i^{j,(k)} - m_{g,i}^{j,(k)*})^2, \quad (4.16)$$

where w_i are design parameters that can be tuned to favor specific DCs. Alternatively, a k-d tree can find the closest feasible allocation in $O(\log |\mathcal{M}^j|)$ average complexity [13].

Fig. 4.3 depicts the system model for graph-job allocation across the three scales. The following theorem guarantees that the convex relaxation is exact, so the continuous solution can be projected back to the discrete feasible set without loss of optimality.

Theorem 1 (Optimality of Convex Relaxation). *The relaxed problem $\mathcal{P3}$ is convex with differentiable affine constraints; hence it satisfies the constraint qualifications implying a zero duality gap. Consequently, the optimal solution of $\mathcal{P3}$ coincides with the optimal solution of the original integer problem $\mathcal{P1}$ after projection onto the feasible discrete set.*

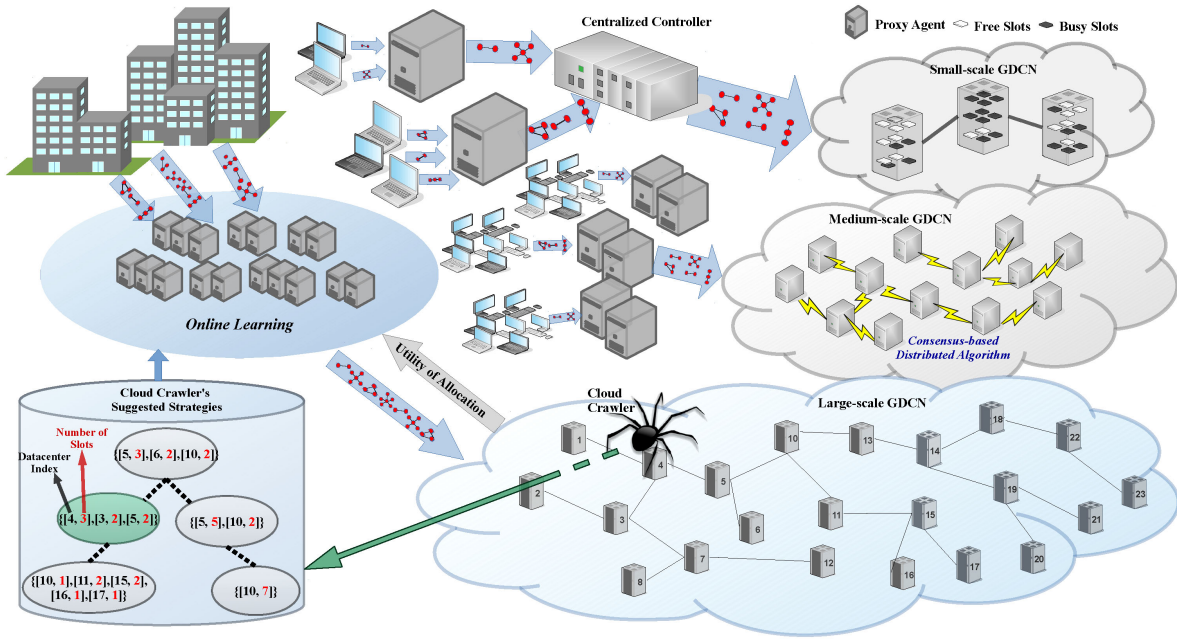


Figure 4.3: System model for graph-job allocation in geo-distributed cloud networks with various scales.

4.2.3 Medium-Scale GDCNs: Distributed CDGA

For medium-scale networks, a centralized processor with global knowledge is impractical and creates a single point of failure. We propose the Consensus-Based Distributed Graph-job Allocation (CDGA) algorithm [13]. The decomposed dual problem at each DC is

$$\max_{\lambda^i \in \mathbb{R}^+, \gamma^i \in \mathbb{R}, \Lambda^i \in \mathbb{R}^+} \sum_{i=1}^{n_d} D^i(\lambda^i, \gamma^i, \Lambda^i), \quad (4.17)$$

where

$$\begin{aligned} D^i(\lambda^i, \gamma^i, \Lambda^i) = & \zeta^i \eta^i N^i \left(\sigma^i \left(\frac{L^i + m_{j,(k)}^{i*}}{|\mathcal{S}^i|} \right)^{\alpha^i} + P_{idle}^i \right) + \zeta^i \nu^i m_{j,(k)}^{i*} \\ & + \lambda^i \left(m_{j,(k)}^{i*} - |\mathcal{S}^i| + L^i \right) + \gamma^i \left(m_{j,(k)}^{i*} - \frac{|\mathcal{V}_j|}{n_d} \right) - \Lambda^i m_{j,(k)}^{i*} \quad \forall i \in \{1, \dots, n_d\}, \end{aligned} \quad (4.18)$$

and the optimal allocation $m_{j,(k)}^{i*}$ is given by

$$m_{j,(k)}^{i*} = \left(\frac{\Lambda^i - \lambda^i - \gamma^i - \zeta^i \nu^i}{\frac{\zeta^i \eta^i N^i \sigma^i \alpha^i}{(|\mathcal{S}^i|)^{\alpha^i}}} \right)^{\frac{1}{\alpha^i - 1}} - L^i, \quad \forall i \in \{1, \dots, n_d\}. \quad (4.19)$$

At each iteration of the CDGA algorithm, each DC first derives the value of the following variables locally using the gradient ascent method:

$$\begin{aligned} \lambda^i(k+1) &= \lambda^i(k) + c_\lambda (\nabla_{\lambda^i} D^i(\lambda^i(k), \gamma^i(k), \Lambda^i(k))), \\ \gamma^i(k+1) &= \gamma^i(k) + c_\gamma (\nabla_{\gamma^i} D^i(\lambda^i(k), \gamma^i(k), \Lambda^i(k))), \\ \Lambda^i(k+1) &= \Lambda^i(k) + c_\Lambda (\nabla_{\Lambda^i} D^i(\lambda^i(k), \gamma^i(k), \Lambda^i(k))), \end{aligned} \quad (4.20)$$

where c_λ , c_γ , and c_Λ are the corresponding step-sizes and γ'^i is a local copy of the global variable. By the envelope theorem, the explicit partial derivatives are

$$\begin{aligned}\nabla_{\lambda^i} D^i(\lambda^i, \gamma'^i, \Lambda^i) &= m_{j,(k)}^{i*} - |\mathcal{S}^i| + L^i, \\ \nabla_{\gamma'^i} D^i(\lambda^i, \gamma'^i, \Lambda^i) &= m_{j,(k)}^{i*} - \frac{|\mathcal{V}_j|}{n_d}, \\ \nabla_{\Lambda^i} D^i(\lambda^i, \gamma'^i, \Lambda^i) &= -m_{j,(k)}^{i*},\end{aligned}\tag{4.21}$$

in which $m_{j,(k)}^{i*}$ is computed from Eq. (4.19) using the current multiplier values. Afterward, the local copies of the global variable are derived by employing the consensus-based gradient ascent method:

$$\gamma^i(k+1) = \sum_{j=1}^{n_d} (\mathbf{W}^\Phi)_{ij} \gamma'^j(k),\tag{4.22}$$

where $\mathbf{W} = \mathbf{I} - \epsilon \mathbf{L}(G_D)$, with $\mathbf{L}(G_D)$ the Laplacian matrix of G_D and $\epsilon \in (0, 1)$, and $\Phi \in \mathbb{N}$ denotes the number of performed consensus iterations among the adjacent DCs.

To see how CDGA works in practice, consider three DCs (d_1 , d_2 , and d_3) that must jointly host a 6-node graph job. Each DC knows only its own electricity price ζ^i , current load L^i , and slot capacity $|\mathcal{S}^i|$; no DC has global knowledge of the others. In the first iteration each DC guesses an initial allocation $m_{j,(k)}^{i*}$ and computes its local Lagrange multipliers λ^i , γ'^i , Λ^i . Because these guesses are made independently, d_1 might think it should host 3 nodes while d_2 thinks it should host only 1. CDGA resolves this disagreement through two alternating steps. First, every DC performs a local gradient ascent on its dual variables using Eqs. (4.20)–(4.21), nudging its allocation toward a locally optimal point that respects its own capacity and power cost. Second, the DCs exchange their local copies of the global consensus variable γ with immediate neighbours and update their estimates via Eq. (4.22). After a few consensus rounds the local estimates $\gamma'^1, \gamma'^2, \gamma'^3$ converge to a common value, meaning all three DCs now agree on how the job should be split. Once convergence is detected, each DC projects its continuous allocation onto the nearest integer vector, yielding a

feasible discrete placement. Algorithm 4 gives the pseudo-code.

Algorithm 4 CDGA: Consensus-Based Distributed Graph-Job Allocation

- 1: **Input:** Convergence threshold $0 < \gamma \ll 1$, maximum iterations K
 - 2: At each DC $d_i \in D$, choose arbitrary initial $\lambda_i(1), g_i(1), L_i(1)$
 - 3: **for** $k = 1$ **to** K **do**
 - 4: At each DC d_i , derive λ_i, g'_i, L_i for next iteration using Eqs. (4.20)–(4.21)
 - 5: At each DC d_i , update g_i using Eq. (4.22)
 - 6: **if** $|g_i(k+1) - g_i(k)| \leq \gamma$ and $|L_i(k+1) - L_i(k)| \leq \gamma$ and $|\lambda_i(k+1) - \lambda_i(k)| \leq \gamma$ and $|g_i(k+1) - g_j(k)| \leq \gamma, 1 \leq i \neq j \leq n_d$ **then**
 - 7: **goto** line 7
 - 8: **end if**
 - 9: **end for**
 - 10: Derive convex-relaxed solution described in Eq. (4.17)
 - 11: Derive allocation using Eq. (4.19)
 - 12: **return** allocation
-

4.2.4 Large-Scale GDCNs: Cloud Crawlers and RMBA

For large-scale GDCNs, the feasible-allocation set becomes prohibitively large (e.g., $\binom{1000}{3} \approx 1.66 \times 10^8$ for a simple triangle graph job in a 100-DC network). Continuous consensus iterations also flood the network with control traffic. To address these challenges, we introduce *cloud crawlers* (CCRs), which are mobile agents that explore the network topology to find physical clusters that are isomorphic to the job's logic graph [13].

A cloud crawler carries structured information while traveling between adjacent DCs, probes the connectivity and status (power usage, load distribution), and provides a set of suggested allocations. The crawler uses a fast decentralized sub-graph extraction method based on a breadth-first traversal with a bounded depth D . Based on Theorem 1 and Corollary 1 in the original paper, the complexity of finding an isomorphic sub-graph is reduced to $O(D)$. Incomplete allocations are stored in an incomplete-allocation list and extended as the crawler visits new DCs. Completed allocations are inserted into a binary search tree keyed by their estimated total cost.

Cloud Crawlers and Sub-Graph Isomorphism

Algorithm 5 presents the pseudo-code of the cloud crawler routine (CCR). The crawler maintains an incomplete-allocation (IA) list and a feasibility vector Feas_A while traversing the DC network.

Algorithm 5 Cloud Crawling Routine (CCR)

Require: G_{job_j} , $|\mathcal{SA}_j|$, CCR location d^{j_0} , centre node v_c , diameter D

Ensure: Updated *BST* of suggested allocations

- 1: Initialise *BST* (size limit $|\mathcal{SA}_j|$), *IA* as list-of-lists, Visited $\leftarrow \emptyset$
 - 2: Visited $\leftarrow$ Visited $\cup \{d^{j_0}\}$
 - 3: Feas_A[1] $\leftarrow 1$ ▷ Cor. 1
 - 4: **for** $r = 1$ **to** D **do**
 - 5: Feas_A[$r + 1$] $\leftarrow$ Feas_A[r] + $|N_{v_c}^r|$
 - 6: **end for**
-

Algorithm 5 Cloud Crawling Routine (CCR) (continued)

```

7: while true do
8:   Observe current DC load distribution  $\tilde{L}_i$ 
9:    $IA_{\text{temp}} \leftarrow \emptyset$ 
10:  for each incomplete allocation  $LL \in IA$  do
11:     $Last \leftarrow LL[\text{end}]$ 
12:     $AN \leftarrow Last[4]$ 
13:     $j \leftarrow \text{find}(\text{Feas\_A} == AN) + 1$ 
14:     $SA \leftarrow 0$ 
15:    while  $j \leq D + 1$  do
16:       $SA \leftarrow SA + |N_{v_c}^j|$ 
17:      if  $SA \leq |S_i|$  then
18:         $p \leftarrow \mathbb{E}[\tilde{\pi}_i \mid \tilde{L}_i + SA]$ 
19:        Append  $(d_i, SA, p, AN + SA)$  to  $LL$ 
20:        if  $AN + SA == |V^j|$  then
21:           $Tot\_Cost \leftarrow \text{Find\_Tot\_Cost}(LL)$ 
22:           $BST \leftarrow \text{BST\_Add}(BST, Tot\_Cost, \text{Create\_Alloc}(LL), |SA_j|)$ 
23:        else
24:          Push extended  $LL$  to  $IA_{\text{temp}}$ 
25:        end if
26:      end if
27:       $j \leftarrow j + 1$ 
28:    end while
29:  end for
30:  for  $r \in \text{Feas\_A}$  do
31:    if  $r \leq |S_i|$  then
32:       $p \leftarrow \mathbb{E}[\tilde{\pi}_i \mid \tilde{L}_i + r]$ 
33:       $RS \leftarrow |V^j| - r$ 
34:      if  $RS == 0$  then
35:         $BST \leftarrow \text{BST\_Add}(BST, p, [(d_i, r)], |SA_j|)$ 
36:      else
37:         $IA_{\text{temp}}.\text{append}([(d_i, r, p, RS)])$ 
38:      end if
39:    end if
40:  end for
41:   $IA \leftarrow IA_{\text{temp}}$ 
42:  if all adjacent DCs of  $d_i$  are in Visited then
43:    Visited  $\leftarrow \emptyset$ ; randomly choose adjacent  $d_k$ 
44:  else
45:    Randomly choose unvisited adjacent  $d_k$ 
46:  end if
47:  Visited  $\leftarrow \text{Visited} \cup \{d_k\}$ 
48:  Crawl to  $d_k$  and set  $d_i \leftarrow d_k$ 
49: end while
50: return  $BST$ 

```

To turn a raw isomorphic mapping into a scored, DC-level allocation, CCR relies on three compact helpers. Consider a triangle job: the crawler maps its centre to d_1 and its neighbours to d_2 and d_3 , yielding $g = \{v_c \mapsto d_1, v_1 \mapsto d_2, v_2 \mapsto d_3\}$. The following routines convert this mapping into a power-model-ready allocation vector and retain only the cheapest candidates.

Find_Tot_Cost (Algorithm 6). Once an allocation is known, we must score it. Find_Tot_Cost plugs the allocation vector into the electricity-cost expression, summing the per-slot usage charge and the dynamic power drawn by every occupied DC.

Algorithm 6 Find_Tot_Cost

Require: Allocation $\mathbf{M}_j$, power and pricing parameters

Ensure: Estimated total cost C

- 1: $C \leftarrow 0$
 - 2: **for** each DC d_i with $m_j^i > 0$ **do**
 - 3: $C \leftarrow C + m_j^i \cdot \xi^i v^i$
 - 4: $C \leftarrow C + \xi^i \eta^i N^i (\sigma^i (\frac{L^i + m_j^i}{|S^i|})^{\alpha^i} + P_{idle}^i)$
 - 5: **end for**
 - 6: **return** C
-

Create_Alloc (Algorithm 7). The mapping g tells us “which node goes where,” but the DC-level power model needs a vector $\mathbf{M}_j = [m_j^1, m_j^2, \dots]$ that counts how many nodes are placed in each DC. In the triangle example above, Create_Alloc simply counts one node in d_1 , one in d_2 , and one in d_3 , producing $\mathbf{M}_j = [1, 1, 1]$.

Algorithm 7 Create_Alloc

Require: Isomorphic mapping g , graph job $Gjob_j$

Ensure: Allocation vector $\mathbf{M}_j$

- 1: $\mathbf{M}_j \leftarrow \mathbf{0}$
 - 2: **for** each node $v \in \mathcal{V}_j$ **do**
 - 3: $d \leftarrow g(v)$
 - 4: $m_j^{\text{idx}(d)} \leftarrow m_j^{\text{idx}(d)} + 1$
 - 5: **end for**
 - 6: **return** $\mathbf{M}_j$
-

BST_Add (Algorithm 8). The crawler may discover dozens of isomorphic mappings; forwarding all of them would overwhelm the network. BST_Add inserts each (cost, allocation) pair into a size-limited balanced binary search tree, evicting the worst entry when the tree is full. An in-order traversal then yields the strategies in ascending cost order.

Algorithm 8 BST_Add

Require: A binary search tree BST , a *key* and a *value*, the desired size of the suggested strategy set $|\mathcal{SA}_j|$

Ensure: A binary search tree BST

```

1: if  $BST.length < |\mathcal{SA}_j|$  then
2:    $BST = BST.Insert(key, value)$ 
3: else if  $key < BST.get\_max().key$  then
4:    $BST = BST.Delete(BST.get\_max())$ 
5:    $BST = BST.Insert(key, value)$ 
6: end if
7: return  $BST$ 

```

Together, the three routines form a compact pipeline inside the crawler: Create_Alloc translates a raw isomorphic mapping into a DC-level occupancy vector; Find_Tot_Cost turns that vector into a dollar amount; and BST_Add keeps only the cheapest mappings discovered so far. When the crawler finally returns, the proxy agent receives not an unwieldy list of every feasible placement, but a small, sorted set of high-quality candidates ready for the game-theoretic selection stage.

We establish the following theorem and corollary to reduce the complexity of obtaining an isomorphic sub-graph for a CCR from exponential to linear in the diameter D of the graph job.

Theorem 2. Consider graphs G and H with vertex sets $\mathcal{V}_G$ and $\mathcal{V}_H$, respectively, where $|\mathcal{V}_G| \leq |\mathcal{V}_H|$. Assume that H can be partitioned into multiple complete sub-graphs $h_0, \dots, h_N$, $N \geq 1$, with vertex sets $\mathcal{V}_{h_0}, \dots, \mathcal{V}_{h_N}$, where $\cup_{i=0}^N \mathcal{V}_{h_i} = \mathcal{V}_H$, and all the nodes in each pair of sub-graphs with consecutive indices are connected to each other. Consider node $v \in \mathcal{V}_G$ and let $D \leq N$ denote the length of the longest shortest path between v and nodes in $\mathcal{V}_G$. Define $\mathcal{N}_v^k \triangleq \{\hat{v} \in \mathcal{V}_G : SP(\hat{v}, v) = k\}$, $\mathcal{N}_v^0 = \{v\}$, where $SP(\cdot, \cdot)$ denotes the length of

the shortest path between the two input nodes. Let $\{i_j\}_{j=0}^D$ be a sequence of integer numbers that satisfy the following conditions:

$$0 \leq i_0, i_1, i_2, \dots, i_D \leq D + 1, \quad (4.23)$$

$$\sum_{l=0}^D i_l = D + 1, \quad (4.24)$$

$$i_j = 0 \Rightarrow i_{j+1} = 0, \quad \forall j \in \{0, \dots, D - 1\}. \quad (4.25)$$

For such a sequence $\{i_j\}_{j=0}^D$, there is at least an isomorphic sub-graph to G , called G' , in H with the corresponding isomorphism mapping g , for which at least one of the nodes of G' , $v' = g(v)$, belongs to h_0 , if the following set of conditions is satisfied:

$$\left\{ \begin{array}{l} |\mathcal{V}_{h_0}| \geq \sum_{i=0}^{i_0-1} |\mathcal{N}_v^i|, \\ |\mathcal{V}_{h_1}| \geq \sum_{i=1}^{i_1} \mathbf{1}_{\{i_1 \geq 1\}} |\mathcal{N}_v^{i+i_0-1}|, \\ |\mathcal{V}_{h_2}| \geq \sum_{i=1}^{i_2} \mathbf{1}_{\{i_2 \geq 1\}} |\mathcal{N}_v^{i+i_0+i_1-1}|, \\ \vdots \\ |\mathcal{V}_{h_D}| \geq \sum_{i=1}^{i_D} \mathbf{1}_{\{i_D \geq 1\}} |\mathcal{N}_v^{i+\sum_{l=0}^{D-1} i_l-1}|. \end{array} \right. \quad (4.26)$$

Corollary 1. For G_{job_j} , assume a CCR located at DC d^{j_0} allocating at least one node of G_{job_j} , $v_c \in \mathcal{V}_j$, to one slot at d^{j_0} , where the length of longest shortest path between v_c and nodes in $\mathcal{V}_j$ is D . Assume that the CCR's near future path can be represented as $d^{j_0} \rightarrow d^{j_1} \dots \rightarrow d^{j_D}$, where $j_i \neq j_k, \forall i \neq k$. Considering d^{j_i} as h_i in Theorem 2, for each realization of the sequence $\{i_j\}_{j=0}^D$ satisfying Eqs. (4.23)–(4.26), the following described allocation $\mathbf{M}_j =$

$[m_j^1, m_j^2, \dots, m_j^{n_d}]$ is feasible and is isomorphic to G_{job_j} :

$$m_j^k = \begin{cases} \sum_{i=0}^{i_0-1} |\mathcal{N}_{v_c}^i| & k = j_0, \\ \sum_{i=1}^{i_1} \mathbf{1}_{\{i_1 \geq 1\}} |\mathcal{N}_{v_c}^{i+i_0-1}| & k = j_1, \\ \sum_{i=1}^{i_2} \mathbf{1}_{\{i_2 \geq 1\}} |\mathcal{N}_{v_c}^{i+i_0+i_1-1}| & k = j_2, \\ \vdots & \\ \sum_{i=1}^{i_D} \mathbf{1}_{\{i_D \geq 1\}} |\mathcal{N}_{v_c}^{i+\sum_{l=0}^{D-1} i_l-1}| & k = j_D, \\ 0 & \text{Otherwise.} \end{cases} \quad (4.27)$$

By Corollary 1, the complexity of obtaining an isomorphic sub-graph for a CCR is reduced to $O(D)$, where D is the diameter of the graph job.

Strategy Selection: BRMA and RMBA

After the crawlers suggest a small set $\mathcal{SA}_j$ of potentially good strategies, the proxy agents (PAs) must select the best one. Because the utility of a strategy depends on the real-time load, which in turn depends on the strategies chosen by *other* PAs, strategy selection is modeled as a non-cooperative game.

BRMA addresses the exploration-exploitation dilemma by clustering the large strategy set $\mathcal{SA}_j$ via k -means and boosting the reward estimates of unvisited strategies that are similar to already-tried ones. Consecutive iterations are grouped into time-frames $\mathcal{T} = \{tf_1, \dots, tf_{\lceil N/\Gamma \rceil}\}$ of length Γ . Within each frame, the algorithm records how many times every strategy is played:

$$\kappa_m^k = \sum_{n=(k-1)\Gamma+1}^{k\Gamma} \mathbf{1}_{\{A_n=m\}}, \quad m \in \mathcal{SA}_j. \quad (4.28)$$

The selection probability blends exploitation (weight proportional to accumulated reward) with uniform exploration:

$$\mathbf{p}_{j,(m)}(tf_n) = (1 - E) \frac{w_{j,(m)}(tf_n)}{\sum_{a \in \mathcal{SA}_j} w_{j,(a)}(tf_n)} + \frac{E}{|\mathcal{SA}_j|}, \quad (4.29)$$

where $0 < E < 1$ is the exploration rate. Algorithm 9 gives the full procedure.

Algorithm 9 BRMA: Boosted Regret Minimization Assignment**Require:** Length of time-frames Γ , $\mathcal{SA}_j$, number of time frames TN_j .

- 1: Obtain clusters $\mathcal{C} = \{C_1, C_2, \dots, C_{|\mathcal{C}|}\}$ from $\mathcal{SA}_j$ according to [42].
- 2: Assign $w_{j_i}(tf_1) = 1$, $\mathbf{p}_{j_i}(tf_1) = 1/|\mathcal{SA}_j|$, $\forall i \in \mathcal{SA}_j$.
- 3: **for** $n = 1$ to TN_j **do**
- 4: Choose a strategy $m \in \mathcal{SA}_j$ according to $\mathbf{p}_j(tf_{\lceil n/\Gamma \rceil})$ ($A_n = m$) and observe the normalized utility $\tilde{U}_j(n)|_m$.
- 5: **if** $n = \Gamma k$, $k \in \mathbb{Z}^+$ **then**
- 6: Obtain the virtual reward for each $m \in \mathcal{A}_{tf_k}$ as follows:

$$Q'_{j,(m)}(tf_k) = \frac{\sum_{z=(k-1)\Gamma+1}^{k\Gamma} \tilde{U}_j(z)|_{A_z} \mathbf{1}_{\{A_z=m\}}}{\kappa_m^k}. \quad (4.30)$$

- 7: Obtain the virtual reward for those strategies $m \notin \mathcal{A}_{tf_k}$ which at least one strategy from their cluster is chosen as follows:

$$Q'_{j,(m)}(tf_k) = \frac{1}{|C_q|} \left[\sum_{z=(k-1)\Gamma+1}^{k\Gamma} \mathbf{1}_{\{A_z \in C_q\}} \underbrace{\frac{\exp\left(\frac{k_s \langle A_z, m \rangle}{\|A_z\| \|m\|}\right) - 1}{\exp(k_s) - 1}}_{\text{similarity index}} \tilde{U}_j(z)|_{A_z} \right] \text{ if } m \in C_q. \quad (4.31)$$

- 8: For strategy $m \in \mathcal{SA}_j$, $Q'_{j,(m)}(tf_k) = 0$ if neither itself nor any strategy from its cluster is chosen in this time-frame.
- 9: Update the weights of the strategies for the next time-frame:

$$w_{j,(m)}(tf_{k+1}) = w_{j,(m)}(tf_k) \exp\left(\frac{K Q'_{j,(m)}(tf_k)}{|\mathcal{SA}_j|}\right), \quad (4.32)$$

where K is a positive real constant.

- 10: Derive the distribution $\mathbf{p}_j(tf_{k+1})$ according to Eq. (4.29).
- 11: **end if**
- 12: **end for**

BRMA provides a fast surrogate for utility estimation, yet it does not guarantee convergence to a game-theoretic equilibrium. To close this gap we layer a regret-matching mechanism on top of BRMA's suggested strategies. The resulting RMBA algorithm treats every proxy agent as a player in a non-cooperative game whose

payoff is the net utility after paying the DCPs.

The payment of PA p_k for strategy $\mathbf{M}_{j,(a_k)}$ aggregates electricity cost and a penalty for over-subscription:

$$Y(\mathbf{M}_{j,(a_k)}, \{\mathbf{M}_{j,(a_{k'})}\}_{k' \neq k}) = \sum_{i=1}^{n_d} m_{j,(a_k)}^i \xi^i \nu^i + \sum_{i=1}^{n_d} \frac{\xi^i \eta^i N^i (\sigma^i (\frac{L^i + \sum_k m_{j,(a_k)}^i}{|S^i|})^{\alpha^i} + P_{idle}^i)}{\sum_k m_{j,(a_k)}^i} m_{j,(a_k)}^i. \quad (4.33)$$

The PA's utility is the job revenue minus this payment, plus a privacy bonus for using fewer DCs, minus a stiff penalty if any DC is over-capacity:

$$c^{p_k}(\{\mathbf{M}_{j,(a_k)}\}) \triangleq \prod_{i=1}^{n_d} \mathbf{1}_{\{L^i + \sum_k m_{j,(a_k)}^i \leq |S^i|\}} \left(\rho_j - \chi_j Y - \phi_j \sum_i \mathbf{1}_{\{m_{j,(a_k)}^i > 0\}} + \chi_j Y^{max} + \phi_j |\mathcal{V}_j| \right) - \left(1 - \prod_i \mathbf{1}_{\{\cdot\}} \right) \Xi^{p_k}. \quad (4.34)$$

A strategy profile is a correlated equilibrium (CE) if no PA can gain by unilaterally deviating to any alternative strategy, given the mixed strategy of the others. The CE set $\mathcal{CE}_j$ is the convex polytope

$$\mathcal{CE}_j = \left\{ \pi_j : \sum_{\mathbf{M}_{j,(-p_k)}} \pi_j(\cdot) [c^{p_k}(m') - c^{p_k}(m)] \leq 0, \forall p_k, \forall m, m' \in \mathcal{SA}_j \right\}. \quad (4.35)$$

RMBA iteratively updates the strategy distribution via regret matching so that the empirical frequencies converge to a point in $\mathcal{CE}_j$. Algorithm 10 details the update.

Algorithm 10 RMBA: Regret Matching-Based Assignment**Require:** PA p_k , graph job's type j , number of iterations N_j , $\mathcal{SA}_j$.

- 1: Select strategies randomly for the first iteration ($n = 1$).
- 2: **for** $n = 1$ to N_j **do**
- 3: Denote history of allocations up to iteration n as $\{\mathbf{A}_\tau\}_{\tau=1}^n$, where $\mathbf{A}_t$ is a vector consisting of $|\mathcal{P}|$ elements, which the j th one, A_t^j , corresponds to p_j 's used allocation at iteration t .
- 4: Calculate the substituting reward for every two different allocations ($\forall m_1, m_2 \in \mathcal{SA}_j$):

$$SR_{m_1, m_2}^{p_k}(n) = \begin{cases} c^{p_k}(m_2, \mathbf{A}_n^{-k}) & \text{if } A_n^k = m_1, \\ c^{p_k}(\mathbf{A}_n) & \text{O.W.} \end{cases} \quad (4.36)$$

- 5: Calculate the substituting average rewards:

$$\Delta_{m_1, m_2}^{p_k}(n) = \frac{\sum_{\tau=1}^n (SR_{m_1, m_2}^{p_k}(\tau) - c^{p_k}(\mathbf{A}_\tau))}{n}. \quad (4.37)$$

- 6: Calculate the average regret:

$$R_{m_1, m_2}^{p_k}(n) = \max\{\Delta_{m_1, m_2}^{p_k}(n), 0\}. \quad (4.38)$$

- 7: Form the selection probability distribution of the allocations, $\forall m \in \mathcal{SA}_j$, for the next iteration:

$$\begin{cases} p_{j, (m)}^{p_k}(n+1) = \frac{1}{\sum_{m' \in \mathcal{SA}_j} R_{A_n^k, m'}^{p_k}(n)} R_{A_n^k, m}^{p_k}(n) & \text{if } m \neq A_n^k, \\ p_{j, (m)}^{p_k}(n+1) = 1 - \sum_{m' \in \mathcal{SA}_j, m' \neq m} p_{j, (m')}^{p_k}(n+1) & \text{Otherwise.} \end{cases} \quad (4.39)$$

- 8: Choose a strategy for the next iteration $A_{n+1}^{p_k}$ according to the derived distribution.
- 9: **end for**

4.3 Layer III: Joint Task Scheduling and Routing via HPS+

The graph-job allocation layer maps the graph logic but treats the network as a “fixed cost” or a black box. The reality is that WAN bandwidth is shared and fluctuates; an “optimal” map might fail if the route between two DCs is congested. The HPS+ framework jointly optimizes where the task sits (hypergraph partition) and how the data moves (SDN routing) [43].

4.3.1 Augmented Hypergraph Model

Standard hypergraph partitioning treats every vertex as an atomic unit and every net as a set of vertices that must stay together. HPS+ extends this view to capture both task-file dependencies and file-datacenter locality. It first builds an augmented task graph $G_{AG} = (N_{AG}, E_{AG})$ whose vertices are tasks N_T , files N_F , and DCs N_S , and whose edges are the original task-file dependencies plus *metaedges* (f, n) that connect each file to the DCs N_S^f storing its replicas (Fig. 4.4) [43].

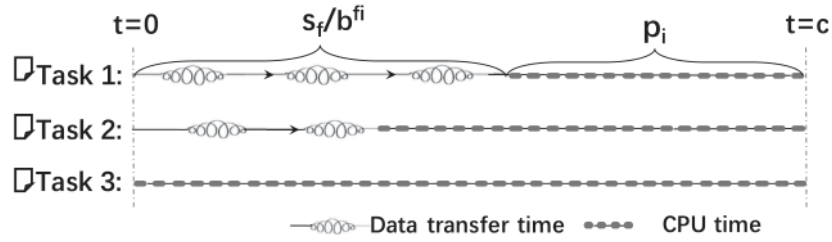


Figure 4.4: Orchestrating the data transfers to balance their completion time.

From G_{AG} HPS+ derives a hypergraph $H = (V, E)$. Real tasks and DCs become ordinary vertices; files become hyperedges (nets) that connect the tasks needing the file to the DCs storing it. To make standard partitioning software produce schedules rather than mere clusters, HPS+ adds four domain-specific twists:

1. **Unit-weighted virtual tasks** (N^v): dummy tasks requiring one core, taking no input, producing no output, and consuming zero execution time. If the total required cores of real tasks is R and the total supply of all DCs is S , the number

of virtual tasks is $S - R$. Their inclusion makes the overall vertex weight equal to the total supply, removing the need for a strict balance criterion.

2. **Negative-weighted datacenter nodes:** the weight of a real task vertex v_t is its required capacity h_{v_t} , while the weight of a DC vertex v_s is $w(v_s) = -R_{v_s}$. File vertices have weight zero.
3. **Double-counting free cut-cost function:** the customized cut cost is

$$\text{Cut}(P) = \sum_{n_j \in P} c(n_j) \cdot (\lambda_j - |N_S^{f_j}|), \quad (4.40)$$

where λ_j is the number of partitions that net n_j connects and $|N_S^{f_j}|$ is the number of DCs storing file f_j . This counts only the transfers to partitions that do not have a local replica.

4. **Zero-approaching balance constraint:** for every partition H_j , the sum of vertex weights satisfies $\sum_{v \in H_j} w(v) \leq \delta$, where δ is a small constant close to zero and less than $\min\{h_{v_t}\}$.

Theorem 2 in the original paper proves that with these settings each partition in a k -way partition (where $k = |N_S|$) contains exactly one datacenter vertex. Theorem 3 proves that the optimal k -way partition minimizing $\text{Cut}(P)$ also minimizes the volume of WAN data transfer [43].

To see how HPS+ turns these abstractions into an actual schedule, consider a simple analytics job with two tasks (T_1, T_2) and two input files (f_1, f_2) distributed across two DCs (S_1, S_2). Suppose f_1 is replicated at both DCs while f_2 resides only at S_1 ; T_1 consumes f_1 and T_2 consumes both f_1 and f_2 . HPS+ first builds the augmented graph: vertices are the two tasks, the two files, and the two DCs; edges connect each task to the files it needs, and meta-edges connect f_1 to both DCs and f_2 to S_1 only. When this graph is turned into a hypergraph, f_1 becomes a net spanning $\{T_1, T_2, S_1, S_2\}$ and f_2 becomes a net spanning $\{T_2, S_1\}$. The multilevel partitioner then coarsens, bisections, and refines. Because f_2 's net strongly pulls T_2 toward S_1 (there is no local replica elsewhere), while f_1 's net is satisfied at either DC, the partitioner

naturally places T_2 with S_1 and T_1 with S_2 . Only f_1 must travel from S_2 to S_1 (for T_2); RBA then gives this flow the bandwidth it needs so that the transfer does not bottleneck the overall makespan. Algorithm 11 summarises the main steps.

Algorithm 11 HPS+ Main Algorithm

- 1: Construct augmented task graph G_{AG} ;
 - 2: Build hypergraph $H = (V, E)$ with four techniques;
 - 3: Partition H into $k = |N_S|$ partitions using MLFM;
 - 4: **for** partition H_j **do**
 - 5: Assign task vertices to unique datacenter vertex $v_s \in H_j \cap N_S$;
 - 6: **end for**
 - 7: **for** task $i \in N_T$ **do**
 - 8: Select replica requiring least hops;
 - 9: **end for**
 - 10: Invoke Algorithm 12;
-

Theorem 3. *With the zero-approaching balance constraint, each partition in a k -way partition (where $k = |N_S|$) contains exactly one datacenter vertex.*

Proof. Consider a k -way partition of the hypergraph H with $k = |N_S|$. The total weight of all vertices is $\sum_{v \in V} w(v) = \sum_{v_t \in N_T} h_{v_t} + \sum_{v_s \in N_S} (-R_{v_s}) + \sum_{v \in N^v} 1 = R - S + (S - R) = 0$. Suppose, for the sake of contradiction, that some partition H_b contains no datacenter vertex. Because there are $k = |N_S|$ partitions and $|N_S|$ datacenter vertices, the pigeon-hole principle forces at least one other partition H_a to contain two datacenter vertices. The combined weight of those two DC vertices alone is $-2R_{v_s} \leq -2 \min_s R_{v_s}$. Even if all virtual-task weight $(S - R)$ were placed in H_a , the partition weight would be at most $-(R_{v_s^{(1)}} + R_{v_s^{(2)}}) + (S - R) + (\text{task weights})$. Since each real-task weight is at least $\min\{h_{v_t}\} > \delta$ and the balance bound is δ , a partition containing two DC vertices necessarily violates $\sum_{v \in H_a} w(v) \leq \delta$ because the two negative DC weights cannot be compensated while keeping the remaining partitions feasible. Hence every partition must contain at least one DC vertex; with exactly $|N_S|$ DC vertices and $|N_S|$ partitions, each partition contains exactly one. The task and virtual vertices are then distributed so that every partition satisfies the balance constraint.

Theorem 4. *The optimal k -way partition minimizing $\text{Cut}(P)$ also minimizes the volume of WAN data transfer.*

Proof. By Theorem 3, each partition H_j contains exactly one DC vertex $v_s^j \in N_S$. A net n_j (corresponding to file f_j) connects λ_j partitions. Among these, $|N_S^{f_j}|$ partitions already store a local replica of f_j . The double-counting-free cut cost for this net is $c(n_j) \cdot (\lambda_j - |N_S^{f_j}|)$, which counts exactly the number of transfers to partitions that do not have a local replica. Minimizing the sum of these terms over all nets is therefore equivalent to minimizing the total number of file transfers across the WAN, i.e., the WAN data transfer volume.

The multilevel framework (MLFM) used for partitioning consists of three phases: (i) *coarsening*, where vertices are successively merged to reduce hypergraph size; (ii) *initial partitioning*, where a small coarsened hypergraph is divided into k parts; and (iii) *refinement*, where the partition is projected back to the original hypergraph and locally improved using vertex-move heuristics. The overall complexity is $O(|N_T| + |N_S| + |N^v| + m|N_F|)$, where m is the number of coarsening levels.

4.3.2 Routing and Bandwidth Allocation (RBA)

After task placement, HPS+ selects the replica requiring the fewest hops as the input for each task. It then orchestrates data transfers using the task-aware Routing and Bandwidth Allocation (RBA) algorithm [43]. RBA balances the allocated bandwidth among data transfers to reduce the makespan. The key insight is that tasks with longer CPU time should receive more bandwidth (unless their input sizes are small) so that their longer computation does not become the bottleneck for makespan.

For a target makespan c , the required bandwidth for flow f_i is derived from

$$b_{fi} = \frac{s_f}{c - p_i}, \quad (4.41)$$

where p_i is the CPU time of task i and s_f is the file size. The algorithm initializes c to a large value

$$c_{\max} = \frac{\max_f s_f}{\min_{uv} R_{uv}} \cdot |E_V| + \max_i p_i, \quad (4.42)$$

and computes the initial bandwidth for each flow using Eq. (4.41), allocating it along the shortest available path. Every time the makespan target is tightened from a previous value c_{old} to a smaller value c , the allocated bandwidth to each flow increases by

$$\Delta b_{ij} = \frac{s_f}{c - p_i} - \frac{s_f}{c_{\text{old}} - p_i}, \quad (4.43)$$

which follows directly from Eq. (4.41). Algorithm 12 gives the full pseudo-code.

Algorithm 12 Routing and Bandwidth Allocation Algorithm (RBA)

```

1: Initialize  $c \leftarrow \text{MAX}$ ;
2: for data flow  $ij$  do
3:    $ij.\text{curP} \leftarrow P_{ij}[0]$ ;
4:   Compute  $b_{ij}$  using Eq. (4.41);
5:   Add  $b_{ij}$  to path used_bw; Subtract from link remaining_bw;
6: end for
7: repeat
8:   Decrease  $c$ ; Compute  $\Delta b_{ij}$  using Eq. (4.43);
9:   for unfinished flow  $ij$  do
10:    if current path cannot satisfy  $\Delta b$  then
11:       $ij.\text{curP} \leftarrow ij.\text{curP} + 1$ ; ▷ next shortest path
12:    if no more available bw then
13:      move to finished set
14:    end if
15:  end if
16: end for
17: until No more bw left or all demands met

```

The time complexity of RBA is $O(f \cdot |E_V|)$, and the overall time complexity of HPS+ is $O(|N_T| + |N_S| + |N^v| + m|N_F| + f|E_V|)$, where m is the number of levels in the multilevel partitioning [43].

Chapter 5

Performance Evaluation

The following sections evaluate each layer independently. Table 5.1 gives a side-by-side summary of the simulators, topologies, workloads, baselines, and key metrics used in the three original studies.

Table 5.1: Summary of Experimental Settings Across the Three Proposed Methods

Aspect	MOSOS (This work)	Graph-Job Allocation (This work)	HPS+ (This work)
Simulator	CloudSim 3.0.3	Custom large-scale GDCN simulator	Custom event-driven simulator
Topology	Single-cluster Amazon EC2 VMs	5–200 DCs, fully-connected or scale-free	China-VO (5 DCs, 7 links) and Google B4 (12 DCs, 19 links)
Workload	Synthetic 100–600 tasks; real traces up to 773k tasks	Graph jobs with 3–5 node topologies; arrival rates 1–4 per type	Up to 2000 tasks; 2000–8000 files; CCR {0.1, 1, 10}
Baselines	FCFS, PSO, MOGA, PBACO	Greedy 1, Greedy 2, exhaustive IP	GS, CDS, HPS, Flutter
Key metrics	Makespan, monetary cost, resource availability, rounds	Incurred power, electricity cost, utility gain	WAN transfer volume, makespan, running time

The results are organised by layer, starting with the single-cluster scheduler.

5.1 Experimental Results: MOSOS

Our MOSOS experiments were conducted on an Intel Core i5 2.6 GHz machine with 4 GB RAM, using the CloudSim 3.0.3 toolkit [4]. Two scenarios were used:

- **Synthetic scenario:** tasks numbered 100–600 with lengths 400–1000 and file sizes 200–1000; VMs were Amazon EC2 instances (m5.xlarge to m5.24xlarge) with prices from \$0.192/h to \$4.608/h.
- **Real-trace scenario:** we used standard workload logs (CEA-Curie-2011-2.1 with 773,138 tasks, CTC-SP2-1996-2.1 with 79,302 tasks, LANL-CM5-1994-3.1 with 201,387 tasks, LCG-2014-2 with 188,041 tasks, NASA-iPSC with 42,264 tasks, and UniLu-Gaia with 51,987 tasks) to mimic real cloud request patterns.

We ran MOSOS against FCFS, PSO, MOGA, and PBACO. Four trends stand out:

- *Makespan.* MOSOS consistently finishes tasks faster than all baselines; its makespan curve lies strictly below those of MOGA, PBACO, and FCFS.
- *Cost.* Total VM-reservation cost under MOSOS is lower than under FCFS and MOGA.
- *Stability.* Box-plots over ten independent runs show tight boxes for MOSOS (low variance) and tall boxes for MOGA (high variance).
- *Dynamic behaviour.* In the on-line setting, MOSOS completes a comparable number of allocation rounds to MOGA while offering shorter response times and higher resource availability. The synthetic-scenario results are plotted in Fig. 5.1, and the stability of makespan and cost across ten independent runs is shown in Figs. 5.2 and 5.3, respectively.

Table 5.2 quantifies the on-line dynamic behaviour of the four algorithms on six standard workload logs. The column R counts resource-allocation rounds; θ is the average number of arriving tasks per round; δ is the mean resource availability ($\frac{1}{R} \sum_{k=0}^R \theta^k$); Δ is the average inter-allocation time; ρ is the average number of selected tasks per round; and the last column shows the mean percentage of released

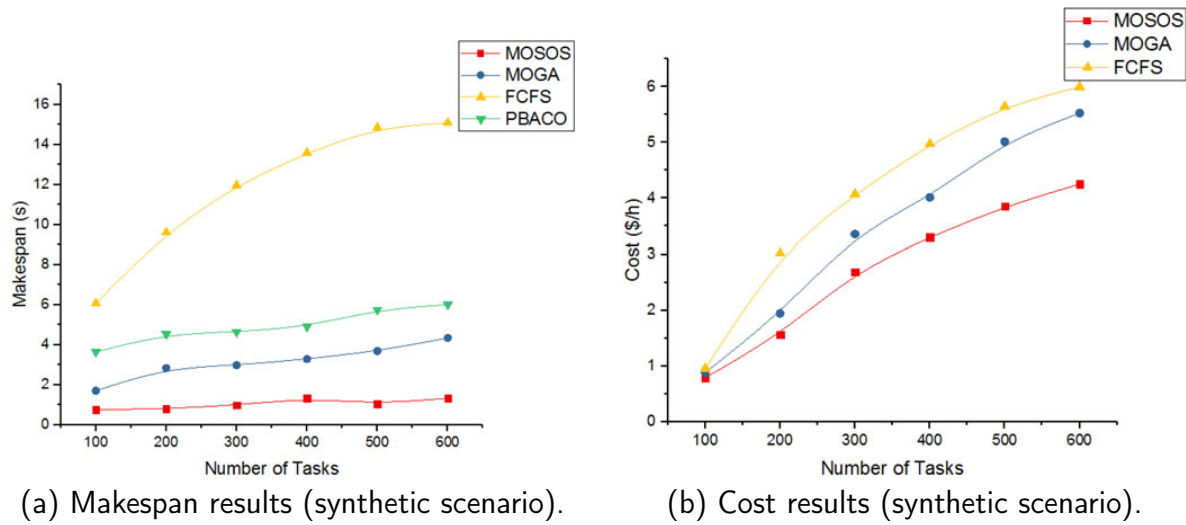


Figure 5.1: Performance comparison of MOSOS, MOGA, FCFS, and PBACO on the synthetic task set.

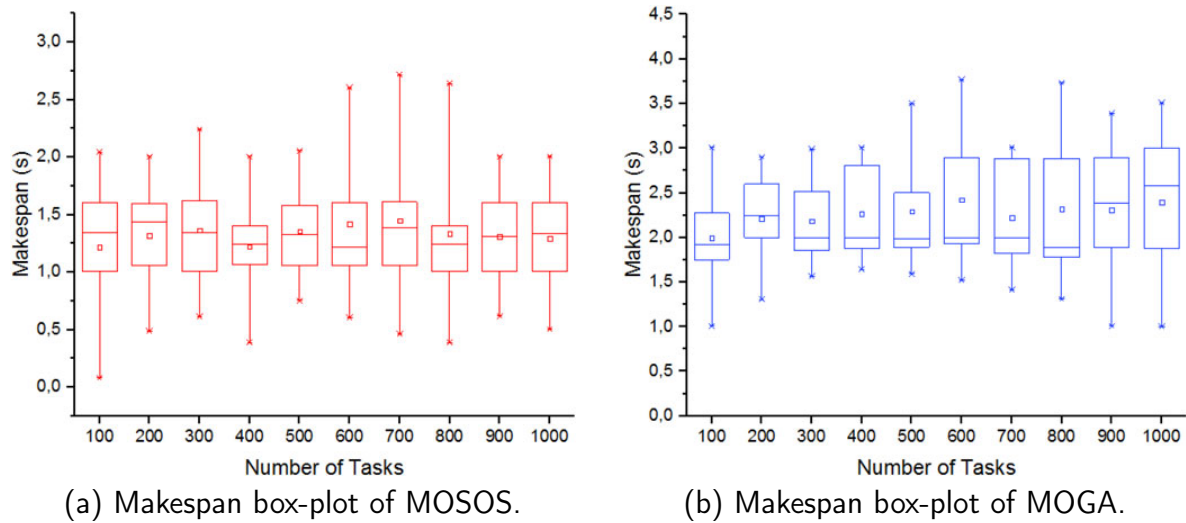


Figure 5.2: Stability analysis of makespan for MOSOS and MOGA over ten independent runs.

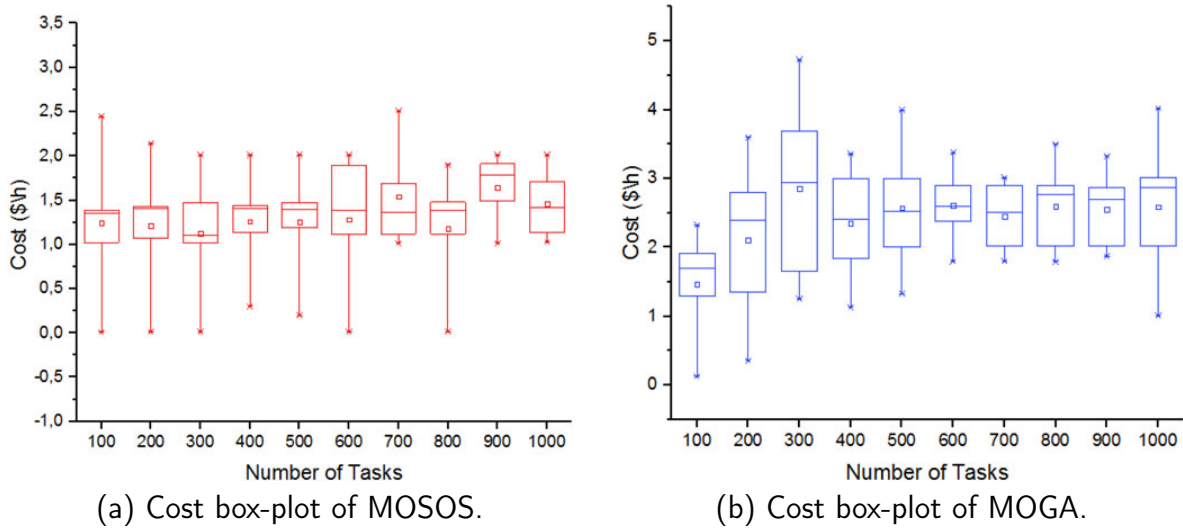


Figure 5.3: Stability analysis of reservation cost for MOSOS and MOGA over ten independent runs.

(residual) resources. MOSOS achieves a comparable number of rounds to MOGA while maintaining shorter response times (lower Δ) and higher resource availability (higher residual percentage) than PSO and FCFS.

Having established that MOSOS handles independent tasks efficiently inside a single cluster, we now turn to the geo-distributed setting where tasks are no longer independent but form graph jobs with communication constraints.

5.2 Experimental Results: Graph-Job Allocation

For our power-aware graph-job allocation framework, we simulate small-, medium-, and large-scale GDCNs with dynamic DC pricing [13]. The power parameters are based on the IBM BladeCenter server: idle power $P^{\text{idle}} = 150$ W, peak power $P^{\text{max}} = 250$ W, PUE $h = 1.3$, exponent $a = 3$, and electricity cost $\lambda_i = \$0.12/\text{kWh}$.

Small scale (5 fully-connected DCs). We tested three scenarios with slots per DC $\{9, 12, 15, 18, 21\}$, $\{12, 15, 18, 21, 24\}$, and $\{15, 18, 21, 24, 27\}$. Our centralized convex approach shows a negligible gap versus exhaustive integer programming search.

Table 5.2: Dynamic model evaluation for FCFS, PSO, MOGA, and MOSOS using benchmark datasets.

Algorithm	LOG	R	θ	δ	Δ	ρ	Res. (%)
FCFS	LANL-CM5-1994-3.1-cln	13	6.3820	19392321321.615	3.334	9389.308	32.6
	LCG	23	14.906	522634766	29.018	8175.739	27.1
	NASA-iPSC	1	0	111625871	13.834	18239	100
	UniLu-Gaia-2014-2	5	0.765	1191264802.4	110.890	10397.6	38.6
	CTC-SP2-1996-2.1-cln	8	3.597	925343434.286	56.461	10332.571	34.2
	CEA-Curie-2011-2.1-cln	37	16.135	416503151.216	8.844	8454.783	28.1
PSO	LANL-CM5-1994-3.1-cln	5	0.793	2322272442.333	32629.224	25741	9.84
	LCG	7	0.528	2133381442.556	27032.540	22964.333	9.75
	NASA-iPSC	5	1.496	5041786652.2	58561.325	24412.2	9.64
	UniLu-Gaia-2014-2	2	0.021	2978162006	40691.068	25994	9.90
	CTC-SP2-1996-2.1-cln	5	0.759	1191264802.4	105.994	10397.6	38.6
	CEA-Curie-2011-2.1-cln	3	0.793	2322272442.333	32629.224	25741	98.1
MOGA	LANL-CM5-1994-3.1-cln	13	6.146	1939148712.385	31.086	9389.307	33.0
	LCG	24	7.128	39432853.229	8.735	7471.610	25.9
	NASA-iPSC	1	0	111625871	8.201	18239	100
	UniLu-Gaia-2014-2	4	0.808	1489081003	14748.907	12997	45.0
	CTC-SP2-1996-2.1-cln	8	3.160	870852165.875	48.726	9652.875	32.9
	CEA-Curie-2011-2.1-cln	38	15.339	405542541.974	8.916	8232.289	27.8
MOSOS	LANL-CM5-1994-3.1-cln	11	4.882	210083598.417	604.276	10171.75	35.2
	LCG	23	14.679	500858317.417	28.775	7835.083	26.8
	NASA-iPSC	1	0	111625871	1430.017	18239	100
	UniLu-Gaia-2014-2	3	0.808	1489081003	14748.907	12997	45.0
	CTC-SP2-1996-2.1-cln	3	0.808	1489081003	14748.907	12997	45.0
	CEA-Curie-2011-2.1-cln	6	2.453	995259618.143	4391.331	11031.857	39.0

On average it saves approximately \$1,100 on electricity cost compared to greedy baselines after 100 iterations, as shown in Fig. 5.4.

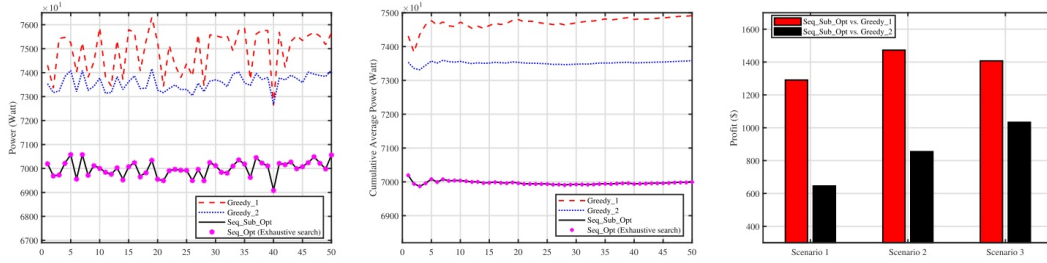


Figure 5.4: Simulation results of the small-scale GDCN: (a) incurred power, (b) cumulative average power, and (c) profit comparison versus greedy baselines.

Medium scale (15 fully-connected DCs). CDGA converges to a unified global variable g across DCs. The cumulative profit after 100 iterations yields an average saving of approximately \$10,000 over greedy algorithms; the convergence trajectories and medium-scale power results are plotted in Figs. 5.5 and 5.6, respectively.

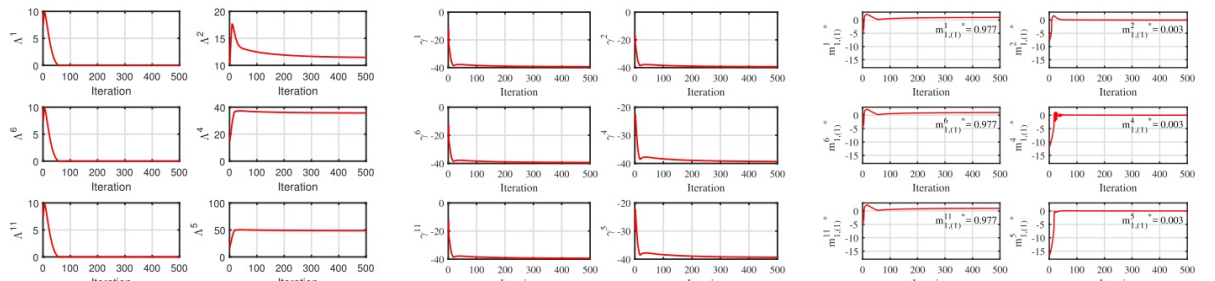


Figure 5.5: Evolution of CDGA parameters at six sampled DCs: (a) local variable Λ , (b) global variable γ , and (c) offered slots.

Large scale (200 DCs, scale-free topology). Cloud crawlers extract up to $|SA_j| = 1000$ suggested strategies. RMBA achieves at least 20% utility gain and lower power consumption compared to random selection, as depicted in Figs. 5.7 and 5.8 [13].

The graph-job allocation layer determines where each sub-task runs, yet it treats the WAN as a black box. The final layer therefore adds coordinated routing, jointly optimising task placement and data movement across the wide-area network.

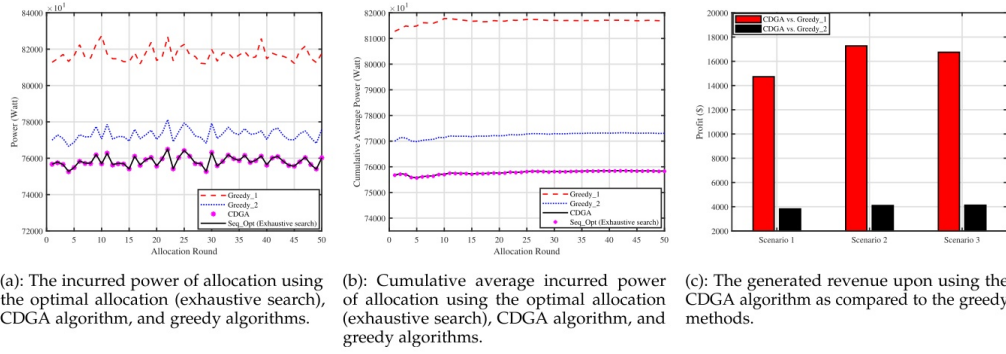


Figure 5.6: Simulation results of the medium-scale GDCN: (a) incurred power, (b) cumulative average power, and (c) profit comparison versus greedy baselines.

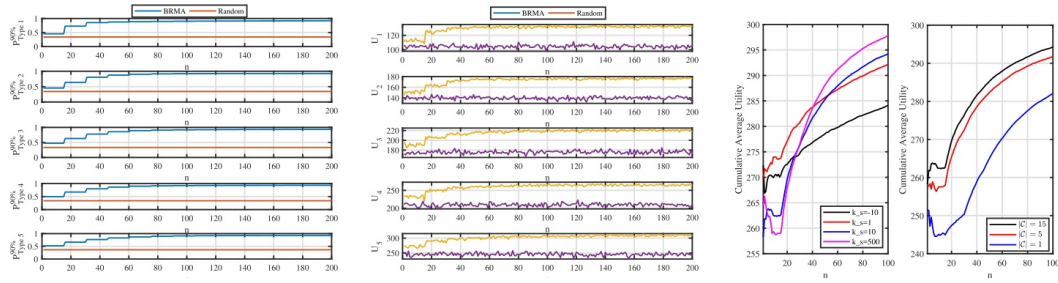


Figure 5.7: Simulation results of the large-scale GDCN with fixed prices: (a) $P^{90\%}$ convergence, (b) utility comparison, and (c) effect of similarity coefficient and number of clusters.

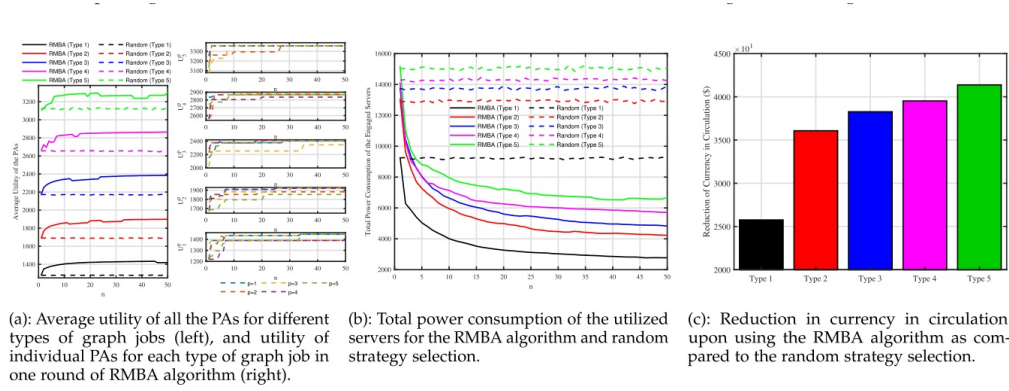


Figure 5.8: Simulation results of the large-scale GDCN with dynamic prices: (a) average utility, (b) total power consumption, and (c) reduction in currency circulation.

5.3 Experimental Results: HPS+

We evaluate HPS+ on two geo-distributed topologies [43]:

- **China-VO model:** 5 data centers (Beijing, Nanjing, Shanghai, Yunnan, Xinjiang) connected by 7 WAN links with 1 Gbps bandwidth. Storage capacities are 55, 50, 30, 10, and 10 TB; core counts are 900, 690, 850, 350, and 160; core ratings are 1330, 1166, 1200, 1140, and 1000 MIPS.
- **Google B4 model:** 12 data centers and 19 WAN links, replicating the real backbone topology of Google. Computational capacities are drawn uniformly from $[2, 32] \times 10^3$ cores, and link bandwidths are set to 1 Gbps.

The workload is based on real astronomical-data-processing scenarios:

- Number of tasks: up to 2000.
- CPU core occupancy per task: randomly generated in $[1, 10]$.
- CPU execution time: randomly generated in $[10, 1000]$ seconds (over 90% of tasks finish within 1000 s, following Facebook’s observation).
- Communication-to-Computation Ratio (CCR): three scenarios $\{0.1, 1, 10\}$.

Table 5.3: Scheduling Results on the China-VO Motivational Example

Algorithm	WAN Transfer Volume	Makespan
Greedy Scheduling (GS)	2800	120
CDS	2800	120
HPS	3800	120
Flutter (shared BW)	2800	120
HPS+	2000	104.3

- Number of files: 2000 for CVO, 8000 for Google B4.
- File size: uniform distribution [0.2 GB, 5 GB].

We compared HPS+ against Greedy Scheduling (GS), Community-Detection-based Scheduling (CDS), the original HPS, and Flutter. The scheduling results on the China-VO example are summarized in Table 5.3. Across the full simulation sweep, HPS+ on average reduces the amount of data transfers by 45–53% and obtains 33–39% better makespan than existing mechanisms [43].

On the Google B4 topology, HPS+ consistently improves makespan (Fig. 5.11). At low data intensity ($CCR = 0.1$) the gains over GS, CDS, HPS, and Flutter average 11%, 15%, 25%, and 10%. When tasks are highly data-intensive ($CCR = 10$), the advantage widens: HPS+ reduces makespan by up to 30% on B4 and by up to 39% on the China-VO topology (Fig. 5.12). HPS suffers because its load-balancing objective inflates transfer volume; CDS beats HPS but its margin shrinks as the task count grows; Flutter matches GS because its exclusive-bandwidth assumption is unrealistic in shared networks. The WAN transfer volume and makespan results under varying CCR are shown in Figs. 5.9–5.12.

We used LocalSolver on the CVO network (problem size: 1700 tasks, 2000 files, $> 200,000$ variables, 73,000 constraints). The optimal MIP solution takes over 20 hours. At $CCR = 0.1$, HPS+ generates on average 10.6% *less* WAN data transfer than the optimal (because the optimal minimizes makespan, not transfer volume). At $CCR = 1$ and 10, HPS+ generates more WAN data transfer than the optimal by averages of 14.5% and 18.3%, respectively. The average makespan gap between

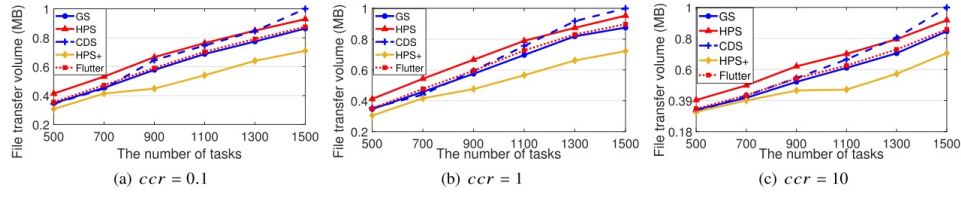


Figure 5.9: Total data transfer volume generated by algorithms in the B4 network under different CCR settings (normalized to ≤ 1).

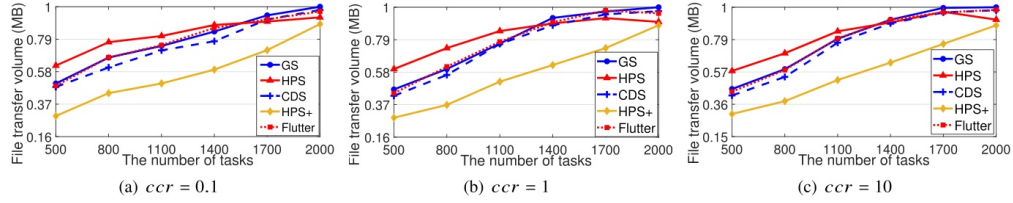


Figure 5.10: Total data transfer volume generated by algorithms in the CVO network under different CCR settings (normalized to ≤ 1).

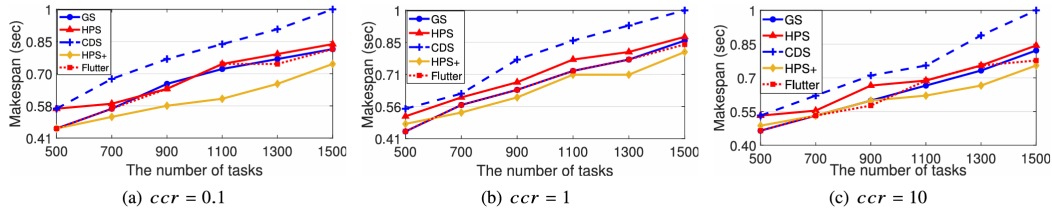


Figure 5.11: Makespan generated by algorithms in the B4 network under different CCR settings (normalized to ≤ 1).

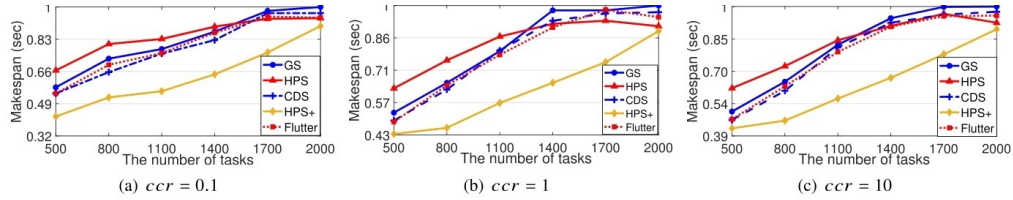


Figure 5.12: Makespan generated by algorithms in the CVO network under different CCR settings (normalized to ≤ 1).

HPS+ and the optimal increases slightly from 11% to 16.8% as CCR rises from 0.1 to 10, as visualized in Fig. 5.13.

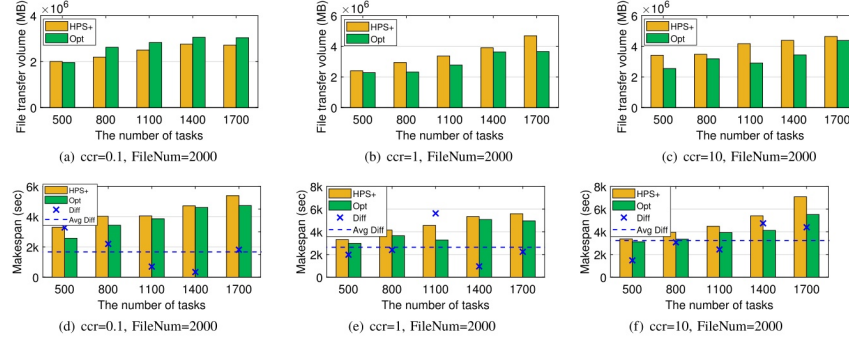


Figure 5.13: Comparison between HPS+ and the optimal MIP solution on the CVO network (FileNum = 2000). Top row: file transfer volume; bottom row: makespan.

In terms of running time, Flutter is prohibitively slow, reaching 6,089 seconds for 2000 tasks due to ILP solving. GS, CDS, HPS, and HPS+ all finish in less than 4 seconds. HPS+ exhibits stable running time because it eliminates the task-reassigning phase found in HPS and the total hypergraph vertex count remains effectively constant thanks to virtual tasks, as confirmed in Fig. 5.14 [43].

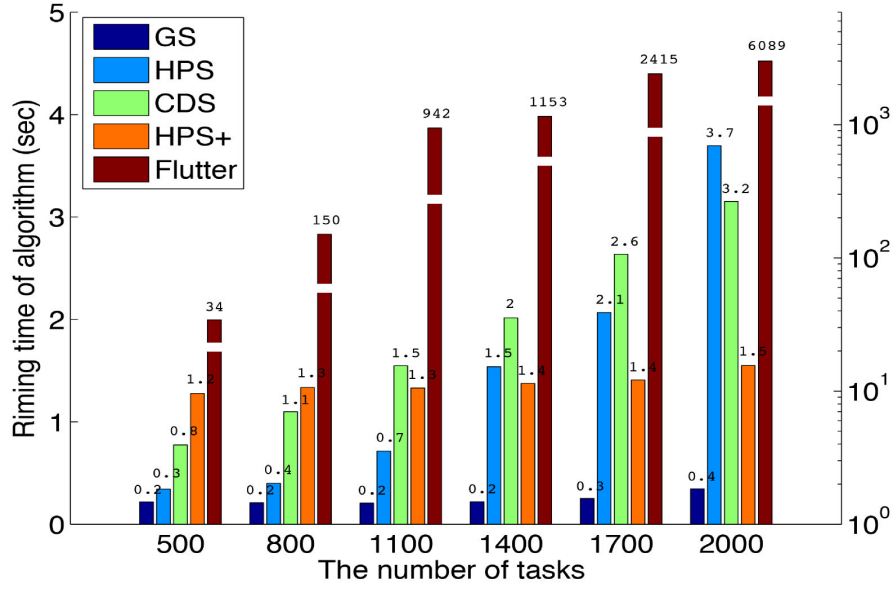


Figure 5.14: Running time of HPS+, GS, CDS, HPS, and Flutter on the CVO network.

The experimental results confirm that each layer improves upon its predecessors: MOSOS outperforms single-cluster baselines in makespan and cost; the graph-job allocator scales from small convex programmes to large crawler-based deployments while respecting power budgets; and HPS+ reduces WAN transfer volume and makespan by co-optimising placement and routing. We now summarise the contributions and discuss practical deployment considerations.

Chapter 6

Conclusion and Future Work

Our three methods form a natural hierarchy: MOSOS addresses the “leaf” problem of assigning independent tasks to VMs in a single cluster; the power-aware graph-job framework lifts the abstraction to geo-distributed DCs and respects job topology; and HPS+ closes the loop by co-optimizing the WAN layer. A future distributed OS could pipeline these three layers:

1. At the lowest level, a MOSOS-like scheduler continuously allocates independent tasks and micro-services to local VMs.
2. At the cluster-to-cluster level, a cloud-crawler-based allocator maps graph jobs onto DCs using real-time power prices and regret-matching equilibrium.
3. At the network level, an HPS+-style SDN controller re-balances bandwidth whenever the task placement or traffic matrix changes.

Several open problems remain. First, all three works assume that task requirements are known in advance or can be estimated accurately; online learning of task lengths and file sizes remains challenging. Second, the power model in Eq. (3.4) abstracts away the idle/boot state of individual servers; a finer-grained server-state model could further improve energy efficiency. Third, link failures and security preferences (e.g., limiting the number of DCs used per job) are only partially addressed by RMBA’s privacy term; resilient scheduling under adversarial or stochastic network conditions is an important direction. Fourth, while MOSOS, CDGA, and HPS+

have each been evaluated independently, there is no published study on their *end-to-end* integration: how should the local scheduler hand off graph jobs to the GDCN allocator, and how quickly can the SDN controller adapt when the task-to-DC mapping changes? Answering these questions requires not only algorithmic research but also systems engineering, including the design of suitable APIs, control-loop latencies, and failure-recovery protocols.

Finally, deploying such an integrated stack on a real production system such as China-VO would require addressing practical issues including fault tolerance, tenant isolation, and incremental reconfiguration. In a multi-tenant cloud, different users may have conflicting quality-of-service requirements; one tenant’s burst of graph jobs could starve another tenant’s latency-sensitive tasks. A future distributed OS will therefore need hierarchical schedulers that enforce service-level objectives while still exploiting the geo-distributed optimization opportunities investigated in this thesis.

6.1 Implementation Considerations

Bridging the gap between theory and a production distributed OS requires attention to several engineering issues. First, *control-loop latency* matters: MOSOS runs in $O(n^2)$ time and can be invoked every few seconds, but CDGA’s consensus iterations may take hundreds of milliseconds per DC, and HPS+ hypergraph partitioning scales with the number of tasks. A practical system would likely run MOSOS on a fast inner loop (seconds), the graph-job allocator on a medium loop (tens of seconds), and the WAN routing optimizer on an outer loop (minutes or upon major job arrivals).

Second, *state synchronization* between layers is non-trivial. The local MOSOS scheduler must inform the GDCN allocator when a graph job is ready to be promoted from a single cluster to the wide-area network; conversely, the GDCN allocator must communicate the chosen DC mapping back to MOSOS so that local VM reservations can be released or re-purposed. Similarly, HPS+ needs the output of the GDCN allocator (which slots in which DCs are occupied) to set up the correct

source-destination pairs for bandwidth allocation.

Third, *failure handling* is essential. If a DC crashes after a graph job has been mapped but before the SDN controller has installed the flows, the system must roll back the allocation and re-invoke the crawler or the consensus algorithm. Checkpointing the state of the MOSOS ecosystem, the CDGA Lagrangian multipliers, and the HPS+ hypergraph partition would enable fast recovery.

Finally, *multi-tenant isolation* requires that the optimizations investigated here respect service-level objectives (SLOs). For example, a tenant running latency-sensitive micro-services should not be displaced by another tenant's batch graph job simply because the latter yields a lower global makespan. Hierarchical schedulers that reserve capacity for high-priority tenants, while using MOSOS/CDGA/HPS+ to optimize the leftover capacity, are a promising direction.

6.2 Conclusion

This thesis proposes a unified three-layer framework for scheduling in distributed operating systems. At the lowest layer, MOSOS dynamically assigns independent tasks to VMs inside a single cluster, trading off makespan against monetary cost. At the middle layer, a family of power-aware graph-job allocators (centralized convex programming for small networks, distributed consensus (CDGA) for medium networks, and cloud crawlers with regret-matching bandits (RMBA) for large networks) map complex jobs onto geo-distributed DCs while respecting topology and power budgets. At the top layer, HPS+ co-optimizes task placement and WAN routing via an augmented hypergraph so that data movement never becomes the bottleneck.

Experiments on CloudSim, a custom GDCN simulator, and real China-VO astronomy workloads show that every layer outperforms its strongest baseline: MOSOS lowers makespan and cost simultaneously; the graph-job allocators cut electricity bills by thousands of dollars compared to greedy strategies; and HPS+ reduces WAN transfer volume by 45–53% while improving makespan by up to 39%. Pipelining the three layers in a production distributed OS is a promising direction for next-generation geo-distributed clouds.

References

- [1] A. Ali and Z. Özkasap, "Spatial and thermal aware methods for efficient workload management in distributed data centers," *Future Gener. Comput. Syst.*, vol. 153, pp. 360–374, Apr. 2024.
- [2] Amazon Services, "Amazon braket: Explore and experiment with quantum computing," 2025. Accessed: May 25, 2025. [Online]. Available: <https://aws.amazon.com/cn/braket/>
- [3] A. C. Ammari, W. Labidi, F. Mnif, H. Yuan, M. Zhou, and M. Sarrab, "Firefly algorithm and learning-based geographical task scheduling for operational cost minimization in distributed green data centers," *Neurocomputing*, vol. 490, pp. 146–162, Jun. 2022.
- [4] A. Belgacem, K. Beghdad-Bey, and H. Nacer, "Dynamic resource allocation method based on symbiotic organism search algorithm in cloud computing," *IEEE Trans. Cloud Comput.*, vol. 10, no. 3, pp. 1714–1725, July–Sept. 2022.
- [5] J. Bi, Z. Yu, and H. Yuan, "Cost-optimized task scheduling with improved deep Q-learning in green data centers," in *Proc. IEEE Int. Conf. Syst., Man, Cybern.*, Prague, Czech Republic, 2022, pp. 556–561.
- [6] A. Borzunov et al., "Distributed inference and fine-tuning of large language models over the internet," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 12312–12331.

- [7] H. Chen, J. Wen, W. Pedrycz, and G. Wu, "Big data processing workflows oriented real-time scheduling algorithm using task-duplication in geodistributed clouds," *IEEE Trans. Big Data*, vol. 6, no. 1, pp. 131–144, Mar. 2020.
- [8] Y. Chen, Q. Yang, S. He, Z. Shi, J. Chen, and M. Guizani, "FTPipeHD: A fault-tolerant pipeline-parallel distributed training approach for heterogeneous edge devices," *IEEE Trans. Mobile Comput.*, vol. 23, no. 4, pp. 3200–3212, Apr. 2024.
- [9] A. Choudhury et al., "MAST: Global scheduling of ML training across geodistributed datacenters at hyperscale," in *Proc. 18th USENIX Symp. Operating Syst. Des. Implementation*, 2024, pp. 563–580. [Online]. Available: <https://www.usenix.org/conference/osdi24/presentation/choudhury>
- [10] P. Clark, "Toshiba achieves quantum communication over commercial fibre without cooling," *Financial Times*, 2025. Accessed: May 27, 2025. [Online]. Available: <https://www.ft.com/content/51a65e45-302c-45fa8bd1-c828a66b012d>
- [11] Google Cloud, "Encrypt workload data in-use with confidential GKE nodes," 2024, Accessed: May 26, 2025. [Online]. Available: <https://cloud.google.com/kubernetes-engine/docs/how-to/confidentialgke-nodes>
- [12] N. Hogade and S. Pasricha, "Game-theoretic deep reinforcement learning to minimize carbon emissions and energy costs for AI inference workloads in geodistributed data centers," 2024, arXiv:2404.01459.
- [13] S. Hosseinalipour, A. Nayak, and H. Dai, "Power-aware allocation of graph jobs in geo-distributed cloud networks," *IEEE Trans. Parallel Distrib. Syst.*, vol. 31, no. 4, pp. 749–765, Apr. 2020.
- [14] M. Hussain, L.-F. Wei, A. Rehman, F. Abbas, A. Hussain, and M. Ali, "Deadline-constrained energy-aware workflow scheduling in geographically distributed cloud data centers," *Future Gener. Comput. Syst.*, vol. 132, pp. 211–222, 2022.

- [15] IBM, "IBM quantum: Explore the quantum future," 2025. Accessed: May 25, 2025. [Online]. Available: <https://quantum.ibm.com/>
- [16] D. Ivanov, J. Smith, J. Wang, A. Petrov, S. Volkov, and D. Zhao, "Optimizing parallel processing with quantum-inspired task scheduling techniques," 2025. Accessed: May 25, 2025. [Online]. Available: https://www.researchgate.net/publication/389324244_Optimizing_Parallel_Processing_with_Quantum-Inspired_Task_Scheduling_Techniques
- [17] S. Khalid and I. Ahmad, "Dual optimization of revenue and expense in geo-distributed data centers using smart grid," *IEEE Trans. Cloud Comput.*, vol. 11, no. 2, pp. 1622–1635, Second Quarter 2023.
- [18] A. Kiani and N. Ansari, "Profit maximization for geographically dispersed green data centers," *IEEE Trans. Smart Grid*, vol. 9, no. 2, pp. 703–711, Mar. 2018.
- [19] C. Li, M. Song, Q. Zhang, and Y. Luo, "Cluster load based content distribution and speculative execution for geographically distributed cloud environment," *Comput. Netw.*, vol. 186, Feb. 2021, Art. no. 107807.
- [20] C. Li, Y. Zhang, Z. Hao, and Y. Luo, "An effective scheduling strategy based on hypergraph partition in geographically distributed datacenters," *Comput. Netw.*, vol. 170, Apr. 2020, Art. no. 107096.
- [21] C. Li, J. Liu, M. Wang, and Y. Luo, "Fault-tolerant scheduling and data placement for scientific workflow processing in geo-distributed clouds," *J. Syst. Softw.*, vol. 187, May 2022, Art. no. 111227.
- [22] X. Li, W. Yu, R. Ruiz, and J. Zhu, "Energy-aware cloud workflow applications scheduling with geo-distributed data," *IEEE Trans. Serv. Comput.*, vol. 15, no. 2, pp. 891–903, Mar./Apr. 2022.

- [23] X. Li, F. Chen, R. Ruiz, and J. Zhu, "MapReduce task scheduling in heterogeneous geo-distributed data centers," *IEEE Trans. Serv. Comput.*, vol. 15, no. 6, pp. 3317–3329, Nov./Dec. 2022.
- [24] X. Lin, J. Wu, A. K. Bashir, W. Yang, A. Singh, and A. A. AlZubi, "FairHealth: Long-term proportional fairness-driven 5G edge healthcare in internet of medical things," *IEEE Trans. Ind. Informat.*, vol. 18, no. 12, pp. 8905–8915, Dec. 2022.
- [25] Microsoft, "Azure quantum: Open ecosystem for quantum innovation," 2025. Accessed: May 25, 2025. [Online]. Available: <https://azure.microsoft.com/en-us/products/quantum/>
- [26] S. Nithyanantham and G. Singaravel, "Resource and cost aware glowworm MapReduce optimization based Big Data processing in geo distributed data center," *Wireless Pers. Commun.*, vol. 117, no. 4, pp. 2831–2852, Apr. 2021.
- [27] S. Nithyanantham and G. Singaravel, "Hybrid deep learning framework for privacy preservation in geo-distributed data centre," *Intell. Automat. Soft Comput.*, vol. 32, no. 3, pp. 1905–1919, 2022.
- [28] H. Okawa, X.-Z. Tao, Q.-G. Zeng, and M.-H. Yung, "Quantumannealing-inspired algorithms for multijet clustering," *Phys. Lett. B*, vol. 864, 2025, Art. no. 139393.
- [29] S. Padhi and R. B. V. Subramanyam, "Uncertainty level-based algorithms by managing renewable energy for geo-distributed datacenters," *Cluster Comput.*, vol. 27, pp. 5337–5354, Jan. 2024.
- [30] Y. Qin, W. Han, Y. Yang, and W. Yang, "Joint energy optimization on the server and network sides for geo-distributed data centers," *J. Supercomputing*, vol. 77, no. 7, pp. 7757–7790, Jul. 2021.
- [31] H. Wang, D. Niu, and B. Li, "Turbo: Dynamic and decentralized global analytics via machine learning," *IEEE Trans. Parallel Distrib. Syst.*, vol. 31, no. 6, pp. 1372–1386, Jun. 2020.

- [32] Y. Wu, S. Tang, C. Yu, B. Yang, C. Sun, J. Xiao, H. Wu, and J. Feng, "Task scheduling in geo-distributed computing: A survey," *IEEE Trans. Parallel Distrib. Syst.*, vol. 36, no. 10, pp. 2073–2087, Oct. 2025.
- [33] T. Xie, C. Li, N. Hao, and Y. Luo, "Multi-objective optimization of data deployment and scheduling based on the minimum cost in geo-distributed cloud," *Comput. Commun.*, vol. 185, pp. 142–158, Mar. 2022.
- [34] J. Xu and B. Palanisamy, "Cost-aware & fault-tolerant geo-distributed edge computing for low-latency stream processing," in *Proc. IEEE 7th Int. Conf. Collaboration Internet Comput.*, Atlanta, GA, USA, 2021, pp. 117–124.
- [35] H. Yuan, H. Liu, and J. Bi, "Revenue and energy cost-optimized biobjective task scheduling for green cloud data centers," *IEEE Trans. Automat. Sci. Eng.*, vol. 18, no. 2, pp. 817–830, Apr. 2021.
- [36] H. Yuan, J. Bi, J. Zhang, and M. Zhou, "Energy consumption and performance optimized task scheduling in distributed data centers," *IEEE Trans. Syst., Man, Cybern. Syst.*, vol. 52, no. 9, pp. 5506–5517, Sep. 2022.
- [37] H. Yuan, J. Bi, and A. C. Ammari, "Biobjective task scheduling for distributed green data centers," *IEEE Trans. Automat. Sci. Eng.*, vol. 18, no. 2, pp. 731–742, Apr. 2021.
- [38] H. Yuan, J. Bi, and M. Zhou, "Energy-efficient and QoS-Optimized adaptive task scheduling and management in clouds," *IEEE Trans. Automat. Sci. Eng.*, vol. 19, no. 2, pp. 1233–1244, Apr. 2022.
- [39] H. Yuan, J. Bi, and M. Zhou, "Profit-sensitive spatial scheduling of multiapplication tasks in distributed green clouds," *IEEE Trans. Automat. Sci. Eng.*, vol. 17, no. 3, pp. 1097–1106, Jul. 2020.
- [40] P. Zhang, X. Ma, Y. Xiao, W. Li, and C. Lin, "Two-level task scheduling with multi-objectives in geo-distributed and large-scale SaaS cloud," *World Wide Web*, vol. 22, no. 6, pp. 2291–2319, Nov. 2019.

- [41] S. Zhang, M. Xu, W. Y. Bryan Lim, and D. Niyato, "Sustainable AIGC workload scheduling of geo-distributed data centers: A multi-agent reinforcement learning approach," in Proc. IEEE Glob. Commun. Conf.. Kuala Lumpur, Malaysia, 2023, pp. 3500–3505.
- [42] J. Zhao, M. A. Rodriguez, and R. Buyya, "A deep reinforcement learning approach to resource management in hybrid clouds harnessing renewable energy and task scheduling," in Proc. IEEE 14th Int. Conf. Cloud Comput.. Chicago, IL, USA, 2021, pp. 240–249.
- [43] L. Zhao, Y. Yang, A. Munir, A. X. Liu, Y. Li, and W. Qu, "Optimizing geo-distributed data analytics with coordinated task scheduling and routing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 31, no. 2, pp. 279–293, Feb. 2020.
- [44] A. C. Zhou, J. Luo, R. Qiu, H. Tan, B. He, and R. Mao, "Adaptive partitioning for large-scale graph analytics in geo-distributed data centers," in Proc. IEEE 38th Int. Conf. Data Eng., Kuala Lumpur, Malaysia, 2022, pp. 2818–2830.
- [45] A. C. Zhou, B. Shen, Y. Xiao, S. Ibrahim, and B. He, "Cost-aware partitioning for efficient large graph processing in geo-distributed datacenters," *IEEE Trans. Parallel Distrib. Syst.*, vol. 31, no. 7, pp. 1707–1723, Jul. 2020.
- [46] A. C. Zhou, Y. Xiao, Y. Gong, B. He, J. Zhai, and R. Mao, "Privacy regulation aware process mapping in geo-distributed cloud data centers," *IEEE Trans. Parallel Distrib. Syst.*, vol. 30, no. 8, pp. 1872–1888, Aug. 2019.